

TECHNICAL SPECIFICATION

LEAD Quiz Portal

Proctored Online Examination System — Full Architecture & Implementation Guide

450 CONCURRENT STUDENTS	5 PROCTOR STATIONS	10 CORE FEATURES	~52 DEV-DAYS
-----------------------------------	------------------------------	----------------------------	------------------------

Document	Architecture & implementation specification
Version	1.3
Date	10 September 2026
Identity model	Roll number + issued password
Status	Draft for review

Contents

Full Technical Specification & Implementation Guide

1. Tech Stack

- 1.1 Recommendation Summary
- 1.2 Why not the alternatives
- 1.3 Repository Layout (monorepo — pnpm workspaces + Turborepo)

2. System Architecture

3. Data Model

- 3.1 Event Type Enum — the vocabulary of the whole system

4. Feature 1 — Authentication & ID/Pass Verification

- 4.1 How it works
- 4.2 Implementation

5. Feature 2 — Answer Secrecy

- 5.1 The rules
- 5.2 Implementation
- 5.3 Where you view the answers after the exam
- 5.4 "Can we store answers in a separate database and grade them later?"
- 5.5 What you cannot do

6. Feature 3 — Fullscreen Lock & Focus Monitoring

- 6.1 Behaviour spec
- 6.2 The Proctor Agent
- 6.3 Server-side authority — the important half
- 6.4 Failure modes to handle before exam day
- 6.5 The hard-stop path — screen violations never go to review

7. Feature 4 — Camera Proctoring with YOLO

- 7.1 Design decision: where does inference run?
- 7.2 Client capture loop
- 7.3 Server-side ML service
- 7.4 Temporal smoothing — the first filter
- 7.5 Human-in-the-loop adjudication — the ML never flags anyone by itself
- 7.6 Capacity math — do this before you buy hardware

8. Feature 5 — Screen Capture / Periodic Screenshots

- 8.1 What is actually possible in a browser
- 8.2 Enforcing "entire screen"
- 8.3 Periodic screenshots (recommended default)
- 8.4 Full screen recording (optional, much heavier)
- 8.5 Storage layout & retention

9. Feature 6 — Microphone / Audio Monitoring

- 9.1 Two-tier approach
- 9.2 Honest limits

10. Feature 7 — Anomaly Event Pipeline

- 10.1 Client — synchronous dual capture

- 10.2 Server — ingest, store, notify
- 10.3 Risk scoring — turn 200 noisy events into one number
- 10.4 The evidence bundle — what you hand to a disciplinary committee

11. Feature 8 — Anti-Screenshot / Screen Leak Prevention

- 11.1 Read this first
- 11.2 What to implement in the web app — layered deterrence
- 11.3 What each layer actually buys you

12. Feature 9 — Issue Flagging & Live Student↔Admin Chat

- 12.1 Behaviour spec
- 12.2 Socket rooms model
- 12.3 Gateway implementation
- 12.4 Admin superpowers on the same channel
- 12.5 Queue UX for 5 stations and 450 students

13. Feature 10 — Admin Proctor Dashboard

- 13.1 Screens
- 13.2 Rendering 450 live tiles without melting the browser
- 13.3 Sorting that matters
- 13.4 Manual unblock by roll number — the release valve

14. Infrastructure, Deployment & Capacity Planning

- 14.1 Sizing for 450 concurrent
- 14.2 Nginx essentials
- 14.3 Required security headers
- 14.4 Reliability requirements — the ones that will save you
- 14.5 Progress checkpointing — a system failure must never cost a student marks or minutes
- 14.6 Load test before exam day — non-negotiable

15. Build Order / Milestones

16. Legal, Consent & Privacy

17. Proposed Additional Features

- 17.1 Stronger anti-cheating
- 17.2 Better student experience
- 17.3 Better admin experience
- 17.4 Longer-term platform features

Appendix A — Reality Check

LEAD Quiz Portal — Proctored Online Examination System

Full Technical Specification & Implementation Guide

Scale: 450 concurrent students · 5 admin proctor stations · ID + password auth · roll-number identity

1. Tech Stack

1.1 Recommendation Summary

Layer	Choice	Why
Frontend	Next.js 15 (React 19, App Router) + TypeScript	SSR for answer secrecy, one codebase for student portal + admin dashboard, API routes for light endpoints
Styling	Tailwind CSS + shadcn/ui	Fast path to an Unstop-grade UI; accessible primitives built in
Client state	Zustand + TanStack Query	The exam is a state machine — far cleaner in Zustand than Redux
Backend API	NestJS (Node 20, TypeScript)	Modules/guards/interceptors map onto auth + proctoring domains; same language as the frontend
Realtime	Socket.IO gateway + Redis adapter	Heartbeats, violations, chat, admin push-commands; Redis adapter allows multiple Node instances
Primary DB	PostgreSQL 16	Relational integrity for students/attempts/answers; JSONB for flexible event payloads
ORM	Prisma	Type-safe queries, painless migrations
Cache / backbone	Redis 7	Sessions, rate limits, socket pub/sub, BullMQ, live presence, atomic locks
Job queue	BullMQ	Async media processing, YOLO inference, evidence bundling
Object storage	MinIO (self-hosted, S3-compatible) or AWS S3	Screenshots / webcam frames / audio — never in Postgres
ML service	Python 3.11 + FastAPI + Ultralytics YOLO11n (ONNX Runtime / TensorRT)	Phone, multiple-person, no-face detection
Face matching	InsightFace (buffalo_l), same FastAPI service	Verifies the person at the keyboard is the enrolled student
Audio analysis	WebRTC VAD (+ optional Silero VAD)	Detect speech during a silent exam
Reverse proxy	Nginx (or Caddy for auto-TLS)	TLS termination, WebSocket upgrade, rate limiting
Deploy	Docker Compose on one strong server → K8s only if you outgrow it	450 users is comfortably single-box territory
Monitoring	Prometheus + Grafana + Loki	You must watch socket count and queue depth live on exam day
Error tracking	Sentry	Client-side exam crashes are the #1 support ticket

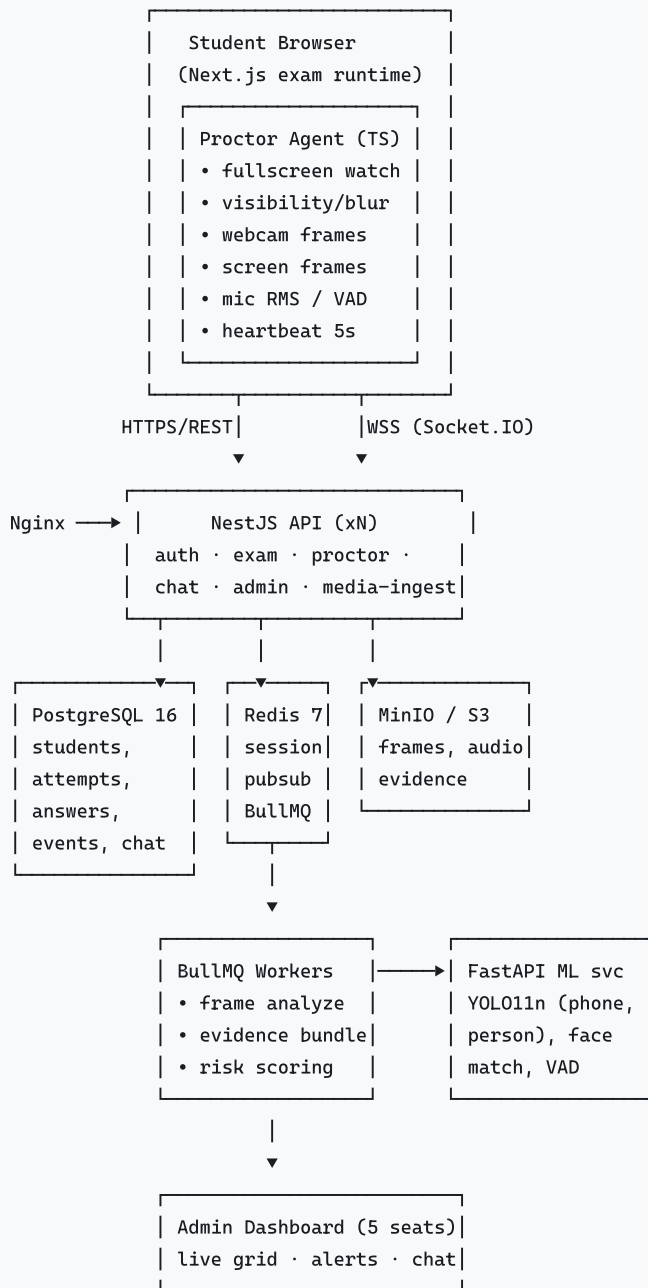
1.2 Why not the alternatives

- **MERN with plain Express** — workable, but you will hand-roll guards, validation, DI and websocket structure that NestJS gives you for free. At this feature count, structure wins.
- **Firebase / Supabase only** — realtime is easy, but you cannot run YOLO, you don't control the media pipeline, and per-event write costs across 450 students × 3 hours of proctoring get ugly fast.
- **Django** — excellent on the ML side, but you'd still split languages across frontend/backend. Keeping ML as a separate FastAPI microservice is the better boundary anyway.
- **Electron / desktop lockdown browser** — *genuinely more secure* (see §11.1), but adds install friction for 450 students. Recommended as Phase 2, or mandatory only for high-stakes rounds.

1.3 Repository Layout (monorepo — pnpm workspaces + Turborepo)

```
lead-quiz/
├── apps/
│   ├── web/           # Next.js - student portal + admin dashboard
│   ├── api/           # NestJS - REST + Socket.IO gateway
│   ├── worker/        # NestJS standalone - BullMQ consumers
│   └── ml/             # Python FastAPI - YOLO, face, audio
├── packages/
│   ├── db/            # Prisma schema + client + migrations
│   ├── shared/        # zod schemas, TS types, event enums
│   └── ui/            # shared React components
├── infra/
│   ├── docker-compose.yml
│   ├── nginx/
│   └── grafana/
└── ARCHITECTURE.md
```

2. System Architecture



Key principle: the browser never *decides* anything security-relevant. It *reports*. The server owns the state machine, the warning counter, the block decision, the timer, and the answers.

3. Data Model

Prisma schema — abridged, but complete enough to migrate.

```
// packages/db/prisma/schema.prisma

model Student {
  id          String  @id @default(cuid())
```

```

rollNumber    String    @unique          // PRIMARY human identifier everywhere
name          String
email         String    @unique
passwordHash  String    // argon2id
enrollFaceVec Bytes?    // 512-d InsightFace embedding
createdAt     DateTime @default(now())
attempts      Attempt[]
chatThread    ChatThread?
}

model Exam {
  id          String    @id @default(cuid())
  title       String
  startsAt    DateTime
  endsAt      DateTime
  durationMin Int
  shuffleQ    Boolean   @default(true)
  shuffleOpts Boolean   @default(true)
  maxWarnings Int       @default(3)
  negativeMark Float    @default(0)
  questions   Question[]
  attempts    Attempt[]
}

model Question {
  id          String @id @default(cuid())
  examId      String
  exam        Exam   @relation(fields: [examId], references: [id])
  order       Int
  type        QType   // MCQ_SINGLE | MCQ_MULTI | NUMERIC | SHORT
  body        String   // markdown/HTML
  imageUrl    String?
  marks       Float   @default(1)
  options     Option[]
  // NOTE: no correct-answer field is ever serialized to a student
}

model Option {
  id          String    @id @default(cuid())
  questionId  String
  question    Question @relation(fields: [questionId], references: [id])
  order       Int
  body        String
  isCorrect   Boolean   // SERVER-ONLY - never selected in student queries
}

model Attempt {
  id          String    @id @default(cuid())
  studentId   String
  examId      String
  student     Student   @relation(fields: [studentId], references: [id])
  exam        Exam      @relation(fields: [examId], references: [id])
  status      AttemptStatus // NOT_STARTED|IN_PROGRESS|SUBMITTED|BLOCKED|AUTO_SUBMITTED
  startedAt   DateTime?
  deadlineAt  DateTime?   // server-computed - single source of truth for the timer
  submittedAt DateTime?
  warningCount Int        @default(0)
  riskScore   Float       @default(0)
  questionOrder Json      // per-student shuffled ordering
  blockedReason String?
  ipAddress   String?
}

```

```

    userAgent    String?
    deviceFp      String?                // FingerprintJS hash
    answers       Answer[]
    events        ProctorEvent[]
    media         MediaAsset[]
    @@unique([studentId, examId])
}

model Answer {
    id            String    @id @default(cuid())
    attemptId     String
    attempt       Attempt   @relation(fields: [attemptId], references: [id])
    questionId    String
    value         Json      // {optionIds:[]} | {text:""} | {number:0}
    isMarked      Boolean   @default(false) // "mark for review"
    answeredAt    DateTime  @default(now())
    revision      Int       @default(0)
    @@unique([attemptId, questionId])
}

model ProctorEvent {
    id            String    @id @default(cuid())
    attemptId     String
    attempt       Attempt   @relation(fields: [attemptId], references: [id])
    type          EventType
    severity       Severity // INFO | LOW | MEDIUM | HIGH | CRITICAL
    weight         Float    @default(0) // contribution to riskScore
    payload        Json     // detector output, bbox, confidence, durations
    clientTs       DateTime
    serverTs       DateTime @default(now())
    webcamShot    String?   // MediaAsset id
    screenShot     String?   // MediaAsset id

    // --- human-in-the-loop adjudication (see §7.5) ---
    source         EventSource // DETERMINISTIC | ML_PROPOSAL | ADMIN
    status          EventStatus @default(ACTIVE) // ACTIVE | PENDING_REVIEW | CONFIRMED | DISMISSED | EXPIRED
    confidence      Float?     // detector confidence that produced the proposal
    reviewerId     String?
    reviewedAt     DateTime?
    verdict         String?    // CHEATING | FALSE_POSITIVE | INCONCLUSIVE
    reviewNote     String?
    appliedWeight   Float @default(0) // what actually hit riskScore after review

    @@index([attemptId, serverTs])
    @@index([status, severity, serverTs]) // drives the pending-review queue
}

enum EventSource { DETERMINISTIC ML_PROPOSAL ADMIN }
enum EventStatus { ACTIVE PENDING_REVIEW CONFIRMED DISMISSED EXPIRED }

model MediaAsset {
    id            String    @id @default(cuid())
    attemptId     String
    attempt       Attempt   @relation(fields: [attemptId], references: [id])
    kind          MediaKind // WEBCAM_FRAME|SCREEN_FRAME|AUDIO_CLIP|ID_PHOTO|SCREEN_RECORDING
    objectKey     String    // {examId}/{rollNumber}/{kind}/{ts}.jpg
    sha256        String
    bytes         Int
    capturedAt    DateTime
    @@index([attemptId, kind, capturedAt])
}

```

```

model ChatThread {
  id          String    @id @default(cuid())
  studentId   String    @unique
  student     Student   @relation(fields: [studentId], references: [id])
  status      ThreadStatus // OPEN | CLAIMED | RESOLVED
  claimedBy   String?    // adminId - prevents two admins replying
  category    String?    // TECHNICAL | QUESTION_ERROR | PERSONAL | OTHER
  priority    Int        @default(0)
  messages    ChatMessage[]
  updatedAt   DateTime   @updatedAt
}

model ChatMessage {
  id          String    @id @default(cuid())
  threadId    String
  thread      ChatThread @relation(fields: [threadId], references: [id])
  senderType  SenderType // STUDENT | ADMIN | SYSTEM
  senderId    String
  body        String
  createdAt   DateTime   @default(now())
  readAt      DateTime?
}

model Admin {
  id          String @id @default(cuid())
  email       String @unique
  name        String
  passwordHash String
  role        AdminRole // PROCTOR | SUPER_ADMIN
  stationId   String?   // "STATION-1".. "STATION-5"
}

model AuditLog {
  id          String    @id @default(cuid())
  actorType   String
  actorId     String
  action      String    // BLOCK_STUDENT | UNBLOCK | GRANT_TIME | VIEW_EVIDENCE | EXPORT
  targetId    String
  meta        Json
  createdAt   DateTime   @default(now())
}

```

3.1 Event Type Enum — the vocabulary of the whole system

```
export enum EventType {  
  // Session lifecycle  
  EXAM_STARTED, EXAM_SUBMITTED, EXAM_AUTO_SUBMITTED, HEARTBEAT_LOST, RECONNECTED,  
  
  // Focus / window  
  FULLSCREEN_EXIT, FULLSCREEN_RESTORED,  
  TAB_HIDDEN, TAB_VISIBLE, WINDOW_BLUR, WINDOW_FOCUS,  
  DEVTTOOLS_SUSPECTED, RESIZE_ANOMALY, SECOND_DISPLAY_DETECTED,  
  
  // Input  
  COPY_ATTEMPT, PASTE_ATTEMPT, CONTEXT_MENU, PRINTSCREEN_KEY,  
  SHORTCUT_BLOCKED, RAPID_ANSWER_PATTERN, IMPOSSIBLE_TYPING_SPEED,  
  
  // Camera  
  NO_FACE, MULTIPLE_FACES, FACE_MISMATCH, LOOKING_AWAY,  
  PHONE_DETECTED, BOOK_DETECTED, CAMERA_BLOCKED, CAMERA_DENIED, CAMERA_STREAM_LOST,  
  
  // Screen  
  SCREEN_SHARE_STOPPED, SCREEN_NOT_FULL_DISPLAY, UNKNOWN_WINDOW_ON_SCREEN,  
  
  // Audio  
  SPEECH_DETECTED, MULTIPLE_VOICES, SUSTAINED_NOISE, MIC_MUTED_OR_DENIED,  
  
  // Network / integrity  
  IP_CHANGED, MULTI_SESSION_ATTEMPT, CLOCK_SKEW, API_TAMPER_SUSPECTED,  
  
  // Admin  
  WARNING_ISSUED, STUDENT_BLOCKED, STUDENT_UNBLOCKED, TIME_GRANTED, ISSUE_FLAGGED  
}
```

4. Feature 1 — Authentication & ID/Pass Verification

4.1 How it works

Three gates before a student sees question 1:

- **Gate A — Credentials.** Roll number + a one-time password you issue (CSV import → printed slips or email). Argon2id hashing.
- **Gate B — Single-session lock.** One active attempt per student. A second login elsewhere is refused.
- **Gate C — Pre-flight system check + identity capture.** Camera, mic, screen-share permission, browser version, network speed, and a face photo matched against the enrolled photo.

4.2 Implementation

Bulk roster import (admin, before exam day)

```
// apps/api/src/students/import.service.ts
import argon2 from 'argon2';
import { customAlphabet } from 'nanoid';
const genPwd = customAlphabet('ABCDEFGHIJKLMNOPQRSTUVWXYZ23456789', 8); // no 0/0/1/I

async importRoster(rows: { rollNumber: string; name: string; email: string }[]) {
  const out = [];
  for (const r of rows) {
    const plain = genPwd();
    await this.prisma.student.create({
      data: {
        rollNumber: r.rollNumber.trim().toUpperCase(),
        name: r.name,
        email: r.email.toLowerCase(),
        passwordHash: await argon2.hash(plain, { type: argon2.argon2id }),
      },
    });
    out.push({ rollNumber: r.rollNumber, password: plain }); // export ONCE to PDF slips
  }
  return out; // never retrievable again
}
```

Login + session binding

```
// Access token : 15-min JWT, kept in memory only (NOT localStorage)
// Refresh token: httpOnly, Secure, SameSite=Strict cookie, 8h, rotating
// Redis session : session:{studentId} -> { jti, deviceFp, ip, socketId }

async login(rollNumber: string, password: string, deviceFp: string, ip: string) {
  const s = await this.prisma.student.findUnique({ where: { rollNumber } });
  if (!s || !(await argon2.verify(s.passwordHash, password)))
    throw new UnauthorizedException('Invalid roll number or password');

  const existing = await this.redis.get(`session:${s.id}`);
  if (existing) {
    const prev = JSON.parse(existing);
    if (prev.deviceFp !== deviceFp) {
      await this.events.log(s.id, EventType.MULTI_SESSION_ATTEMPT, Severity.HIGH, { ip, deviceFp });
      throw new ConflictException('An active session exists on another device. Contact a proctor.');
```

- Rate limit: 5 attempts / 10 min per roll number and per IP ([@nestjsjs/throttler](#) on Redis).
- `deviceFp` from [FingerprintJS](#) (open-source) — enough to tell "same laptop, refreshed" from "friend's laptop".
- Lockout must be manually recoverable: a proctor clears `session:{id}` from the dashboard in one click. You *will* need this on exam day.

Gate C — the preflight page (`/exam/{id}/preflight`)

```
const checks = [
  { id: 'browser',    run: () => /Chrome|Edg/.test(navigator.userAgent) && !isMobile() },
  { id: 'camera',    run: () => navigator.mediaDevices.getUserMedia({ video: { width: 1280, height: 720 } }) },
  { id: 'mic',       run: () => navigator.mediaDevices.getUserMedia({ audio: true }) },
  { id: 'screen',    run: () => navigator.mediaDevices.getDisplayMedia({ video: { displaySurface: 'monitor' } }) },
  { id: 'fullscreen', run: () => document.documentElement.requestFullscreen() },
  { id: 'displays',  run: async () => (await navigator.windowManagement?.getScreenDetails?.())?.screens.length ===
1 },
  { id: 'network',   run: () => measureRtt() < 400 },
  { id: 'identity',  run: () => captureAndMatchFace() }, // POST /proctor/identity-verify
];
```

Identity check hits the ML service:

```
# apps/ml/routers/face.py
@router.post("/verify")
async def verify(file: UploadFile, roll_number: str):
    img = read_image(await file.read())
    faces = face_app.get(img)
    if len(faces) == 0: return {"ok": False, "reason": "NO_FACE"}
    if len(faces) > 1: return {"ok": False, "reason": "MULTIPLE_FACES"}
    enrolled = load_embedding(roll_number) # Student.enrollFaceVec
    sim = cosine(faces[0].normed_embedding, enrolled)
    return {"ok": bool(sim > 0.42), "similarity": float(sim)} # tune on YOUR data
```

Tuning note: 0.42 cosine on `buffalo_1` is a starting point, not a law. Run your own enrollment photos against each other and pick a threshold at roughly 1% false-reject rate. **Never auto-block on a face mismatch** — flag it for human review. Lighting, glasses and skin tone all cause real false positives.

If a student has no enrollment photo, fall back to a **manual ID-card check**: capture a webcam photo with the college ID held up, store it as `ID_PHOTO`, and let an admin approve it from a dashboard queue.

5. Feature 2 — Answer Secrecy

Requirement: a student who inspects the page, reads the JS bundle, opens the Network tab, or dumps `window` must not find correct answers. Answers stay server-side and are viewable only by you, after the exam.

5.1 The rules

1. `Option.isCorrect` never appears in any student response. Enforce with an explicit Prisma `select` — never `include`, never a spread.
2. **Server-side grading only.** The client posts `{ questionId, optionIds }`; the server evaluates. The client never learns right/wrong during the exam.
3. **No "check answer" endpoint exists** while an attempt is active. Results compute after `submittedAt` and release at a time you control.
4. **Per-student option shuffle with a server-held mapping.** Even if two students collude via screenshots, "the answer is B" means different things to each.
5. **Questions are paginated and fetched lazily** — one question (or a batch of five) at a time, so the full paper is never sitting in client memory.

5.2 Implementation

The only student-facing question DTO:

```
// apps/api/src/exam/exam.service.ts
const QUESTION_STUDENT_SELECT = {
  id: true, type: true, body: true, imageUrl: true, marks: true,
  options: { select: { id: true, body: true } }, // <-- isCorrect ABSENT. Never add it.
} satisfies Prisma.QuestionSelect;

async getQuestion(attemptId: string, index: number) {
  const attempt = await this.assertActive(attemptId); // status, deadline, not blocked
  const order = attempt.questionOrder as string[]; // server-generated, per student
  const qid = order[index];
  if (!qid) throw new NotFoundException();

  const q = await this.prisma.question.findUnique({
    where: { id: qid }, select: QUESTION_STUDENT_SELECT,
  });
  // deterministic per-student shuffle, seeded by attemptId - stable across refreshes
  q.options = seededShuffle(q.options, hash(attempt.id + qid));
  return q;
}
```

Belt and braces — a global interceptor that strips the field even if someone slips up:

```
// apps/api/src/common/scrub.interceptor.ts
const FORBIDDEN = ['isCorrect', 'correctOptionIds', 'answerKey', 'explanation',
  'passwordHash', 'enrollFaceVec'];

@Injectable()
export class ScrubInterceptor implements NestInterceptor {
  intercept(ctx: ExecutionContext, next: CallHandler) {
    const isAdmin = ctx.switchToHttp().getRequest().user?.role?.startsWith('ADMIN');
    return next.handle().pipe(map(d => (isAdmin ? d : deepOmit(d, FORBIDDEN))));
  }
}
```

And a CI test that fails the build on a leak:

```
it('never leaks answer keys to students', async () => {
  const res = await request(app).get(`/exam/${examId}/q/0`).set(studentAuth);
  expect(JSON.stringify(res.body)).not.toMatch(/isCorrect|answerKey/i);
});
```

Grading (post-submit worker job):

```

async grade(attemptId: string) {
  const answers = await prisma.answer.findMany({ where: { attemptId } });
  const keys = await prisma.option.findMany({
    where: { question: { examId }, isCorrect: true },
    select: { id: true, questionId: true },
  });
  const keyMap = groupBy(keys, 'questionId');

  let total = 0;
  for (const a of answers) {
    const picked = new Set((a.value as any).optionIds ?? []);
    const correct = new Set(keyMap[a.questionId]?.map(o => o.id) ?? []);
    const exact = picked.size === correct.size && [...picked].every(id => correct.has(id));
    total += exact ? marksOf(a.questionId) : -negativeMark;
  }
  return prisma.result.create({ data: { attemptId, score: total } });
}

```

5.3 Where *you* view the answers after the exam

Two channels, both admin-only:

1. Admin UI — `/admin/exams/{id}/answer-key`, behind `@Roles('SUPER_ADMIN')`, every view written to `AuditLog`.
2. Export — `POST /admin/exams/{id}/export` produces an XLSX (`rollNumber | question | studentAnswer | correctAnswer | marks | flags | riskScore`) plus a JSONL of every `ProctorEvent` and signed links to the evidence media.

If you want the key "in a console/file you can inspect", make that an **admin-only debug page** that calls the admin endpoint — not a client-side embedded object. Anything shipped to a student's browser is public, full stop.

5.4 "Can we store answers in a separate database and grade them later?"

Yes — and that is already the design above. Your instinct is right, so it's worth being precise about *which part of it* actually provides the secrecy, because the two halves of the idea are not equally load-bearing.

The half that does the work: deferred, server-side grading.

Storing the student's submitted answers as raw rows (`Answer.value`) and grading them only after the exam is exactly what §5.2 does. It works because during the exam **no comparison ever happens on the student's machine** — the browser posts "I picked option `ckx91...`" and receives `{ ok: true, saved: true }`. There is nothing in the response, the bundle, memory, or the Network tab to inspect, because rightness was never computed there. Grading runs later against `Option.isCorrect`, which never left Postgres.

So: a student cannot see the answers, correct. ✓

The half that doesn't, by itself: "a separate database."

Physical separation doesn't create the secrecy — it's a *consequence* of grading server-side, not the cause. The failure mode people hit is storing answers separately but still having some endpoint that reads the key and returns a verdict to the client ("was I right?", a progress bar showing score, a "you have 12 correct so far" widget). The moment anything derived from the key reaches the browser, separation has bought nothing. **The rule is not "separate storage" — it is "no key-derived data crosses the wire during the exam."**

Where separation *does* genuinely help — worth doing:

Keep the key in its own store with its own credentials, so a SQL-injection or a leaked read-only connection string on the exam API cannot reach it:

exam_db (hot path)	key_db (cold, isolated)
students, attempts,	question_id → correct[]
answers (raw choices),	marks, rubric
events, chat	
← API user: read/write	← API user: NO ACCESS
	← grader job only,
	separate credential

- The **exam API** (the process students talk to) holds a DB role with **no privileges on `key_db` at all**. Not "doesn't query it" — *cannot*.
- The **grading worker** is a different process with a different credential, runs after `submittedAt`, and is not reachable from any student route.
- Practical version: same Postgres instance, separate schema (`answer_key`), `REVOKE ALL ON SCHEMA answer_key FROM exam_api`. You get the isolation without operating a second database.

```
CREATE SCHEMA answer_key;
CREATE TABLE answer_key.option_key (
  option_id text PRIMARY KEY,
  question_id text NOT NULL,
  is_correct boolean NOT NULL
);
REVOKE ALL ON SCHEMA answer_key FROM exam_api; -- the student-facing role
GRANT USAGE, SELECT ON SCHEMA answer_key TO grader; -- the post-exam worker only
```

This is a real, meaningful hardening: it turns "we were careful in application code" into "the credential physically cannot read it." It makes §5.2's scrub interceptor a second line of defence rather than the only one. Note the tradeoff — you lose the single-`JOIN` grading query and Prisma's type safety across the boundary, so the grader does two reads and matches in memory. At 450 students × 60 questions that's trivial.

What it still does not fix: answers leaking through *timing or behaviour*. If your API is slower for correct answers, or a "marked for review" flag is set differently, that's a side channel. Keep the save endpoint's response byte-identical regardless of what was picked — which it is, if it never touches the key.

Verdict: do it — separate schema, separate credential. But understand that the secrecy comes from *never grading in the browser*; separation is defence-in-depth on top of that, protecting you from your own future bug rather than from the student's DevTools.

5.5 What you cannot do

Client-side obfuscation, WASM-hidden keys, encrypted blobs decrypted in JS — all of these ship the key to the attacker's machine. **Server-side grading is the only real answer.** Don't spend a day on obfuscation.

6. Feature 3 — Fullscreen Lock & Focus Monitoring

6.1 Behaviour spec

Trigger	Severity	Action
Exit fullscreen	HIGH	pause exam, blocking overlay, warning +1 , snapshot
<code>visibilitychange</code> → hidden	HIGH	same as above
<code>window.blur</code> > 2 s	MEDIUM	warning +1, snapshot
Right-click / F12 / Ctrl+Shift+I / Ctrl+U	LOW	block key, log, no warning
Copy / paste / cut in the exam area	MEDIUM	block, log
PrintScreen key	HIGH	log, blur the screen, snapshot
DevTools heuristic fires	HIGH	warning +1
Screen share stopped	CRITICAL	pause until re-shared, warning +1
Heartbeat missing > 30 s	HIGH	mark disconnected; > 5 min → auto-submit or admin decision

Escalation: warning 1 → yellow modal · warning 2 → orange "final warning" · warning 3 → **BLOCKED**, attempt frozen and submitted as-is, admin notified. `Exam.maxWarnings` is configurable. **Only the server increments the counter.**

6.2 The Proctor Agent

```
// apps/web/src/proctor/agent.ts
type Violation = { type: EventType; severity: Severity; payload?: any };

export class ProctorAgent {
  private blurTimer?: number;
  private lastFsExit = 0;

  constructor(private socket: Socket, private attemptId: string, private ui: ExamUI) {}

  start() {
    document.addEventListener('fullscreenchange', this.onFsChange);
    document.addEventListener('visibilitychange', this.onVisibility);
    window.addEventListener('blur', this.onBlur);
    window.addEventListener('focus', this.onFocus);
    document.addEventListener('contextmenu', e => { e.preventDefault(); this.report({ type: 'CONTEXT_MENU',
severity: 'LOW' }); });
    document.addEventListener('copy', e => { e.preventDefault(); this.report({ type: 'COPY_ATTEMPT',
severity: 'MEDIUM' }); });
    document.addEventListener('paste', e => { e.preventDefault(); this.report({ type: 'PASTE_ATTEMPT',
severity: 'MEDIUM' }); });
    document.addEventListener('keydown', this.onKey, true);
    this.startDevtoolsProbe();
    this.startHeartbeat();
  }

  private onFsChange = () => {
    if (!document.fullscreenElement) {
      this.lastFsExit = Date.now();
      this.ui.lockWithOverlay('You left fullscreen. The exam is paused.');
```

```

    this.report({ type: 'FULLSCREEN_EXIT', severity: 'HIGH', payload: { at: this.lastFsExit } });
    this.ui.captureEvidence('FULLSCREEN_EXIT'); // dual snapshot, see §10
  } else {
    this.report({ type: 'FULLSCREEN_RESTORED', severity: 'INFO',
      payload: { awayMs: Date.now() - this.lastFsExit } });
    this.ui.unlock();
  }
};

private onVisibility = () => {
  if (document.hidden) {
    this.report({ type: 'TAB_HIDDEN', severity: 'HIGH' });
    this.ui.captureEvidence('TAB_HIDDEN');
  } else this.report({ type: 'TAB_VISIBLE', severity: 'INFO' });
};

private onBlur = () => {
  this.blurTimer = window.setTimeout(() => {
    this.report({ type: 'WINDOW_BLUR', severity: 'MEDIUM' });
    this.ui.captureEvidence('WINDOW_BLUR');
  }, 2000); // 2s grace kills most false positives
};

private onFocus = () => clearTimeout(this.blurTimer);

private onKey = (e: KeyboardEvent) => {
  const combo = `${e.ctrlKey ? 'Ctrl+' : ''}${e.metaKey ? 'Meta+' : ''} +
    ${e.shiftKey ? 'Shift+' : ''}${e.altKey ? 'Alt+' : ''}${e.key}`;
  const blocked = ['F12', 'Ctrl+Shift+I', 'Ctrl+Shift+J', 'Ctrl+Shift+C', 'Ctrl+U', 'Ctrl+S',
    'Ctrl+P', 'Ctrl+C', 'Ctrl+V', 'Ctrl+X', 'Ctrl+A', 'Meta+Shift+3', 'Meta+Shift+4',
    'Meta+Shift+5', 'Alt+Tab', 'Meta+Tab', 'Ctrl+W', 'Ctrl+T', 'Ctrl+N', 'PrintScreen'];

  if (blocked.some(b => combo === b || combo.endsWith(b))) {
    e.preventDefault(); e.stopPropagation();
    const isPrint = e.key === 'PrintScreen' || combo.includes('Meta+Shift');
    if (isPrint) { this.ui.panicBlur(1500); this.ui.captureEvidence('PRINTSCREEN_KEY'); }
    this.report({ type: isPrint ? 'PRINTSCREEN_KEY' : 'SHORTCUT_BLOCKED',
      severity: isPrint ? 'HIGH' : 'LOW', payload: { combo } });
  }
};

private startDevtoolsProbe() {
  // 1. outer/inner size delta
  setInterval(() => {
    const wGap = window.outerWidth - window.innerWidth;
    const hGap = window.outerHeight - window.innerHeight;
    if (wGap > 200 || hGap > 250)
      this.report({ type: 'DEVTOOLS_SUSPECTED', severity: 'HIGH', payload: { wGap, hGap } });
  }, 3000);

  // 2. toString getter trap - fires only when a console renders the object
  const trap: any = /.;
  trap.toString = () => {
    this.report({ type: 'DEVTOOLS_SUSPECTED', severity: 'HIGH', payload: { probe: 'toString' } });
    return '';
  };
  setInterval(() => { console.debug('%c', trap); console.clear(); }, 4000);

  // 3. debugger-statement timing
  setInterval(() => {
    const t = performance.now();
    // eslint-disable-next-line no-debugger

```

```

    debugger;
    if (performance.now() - t > 120)
        this.report({ type: 'DEVTOOLS_SUSPECTED', severity: 'HIGH', payload: { probe: 'debugger' } });
    }, 7000);
}

private startHeartbeat() {
    setInterval(() => this.socket.emit('hb', {
        attemptId: this.attemptId, t: Date.now(),
        fs: !!document.fullscreenElement, hidden: document.hidden,
        focus: document.hasFocus(), screens: (window as any).screen?.isExtended ?? null,
    }), 5000);
}

private report(v: Violation) {
    this.socket.emit('violation', { attemptId: this.attemptId, clientTs: Date.now(), ...v });
    queueOffline(v); // IndexedDB durability - flushed on reconnect
}
}

```

Re-entering fullscreen requires a user gesture (a browser rule you cannot bypass), so the overlay must carry a button:

```

<Overlay blocking>
  <h2>Exam Paused  Fullscreen Required</h2>
  <p>Warning {warningCount} of {maxWarnings}. This has been recorded and reported to the proctors.</p>
  <Button onClick={() => document.documentElement.requestFullscreen()}>Return to Exam</Button>
  <Button variant="ghost" onClick={openIssueFlag}>I'm having a problem</Button>
</Overlay>

```

6.3 Server-side authority — the important half

```
// apps/api/src/proctor/proctor.gateway.ts
@SubscribeMessage('violation')
async onViolation(@ConnectedSocket() sock: Socket, @MessageBody() dto: ViolationDto) {
  const attempt = await this.svc.assertOwnedAndActive(sock.data.studentId, dto.attemptId);

  // dedupe: identical type within 3s counts once
  const fresh = await this.redis.set(`dedup:${attempt.id}:${dto.type}`, '1', 'EX', 3, 'NX');
  if (fresh === null) return;

  const weight = WEIGHTS[dto.type] ?? 0;
  const ev = await this.prisma.proctorEvent.create({
    data: { attemptId: attempt.id, type: dto.type, severity: dto.severity, weight,
      payload: dto.payload ?? {}, clientTs: new Date(dto.clientTs) },
  });

  let { warningCount, riskScore } = attempt;
  if (WARNABLE.has(dto.type)) warningCount += 1;
  riskScore += weight;

  const maxW = attempt.exam.maxWarnings;
  const blocked = warningCount >= maxW;

  await this.prisma.attempt.update({
    where: { id: attempt.id },
    data: {
      warningCount, riskScore,
      ...(blocked && { status: 'BLOCKED',
        blockedReason: `Exceeded ${maxW} warnings (${dto.type})` }),
    },
  });

  sock.emit(blocked ? 'blocked' : 'warning', { warningCount, maxWarnings: maxW, reason: dto.type });
  this.server.to('admins').emit('live:event',
    { rollNumber: attempt.student.rollNumber, ...ev, warningCount, riskScore });

  if (blocked) await this.evidence.bundle(attempt.id, ev.id); // freeze a case file
}
```

WARNABLE contains only deterministic, non-ML signals — things the browser observed as fact, not things a model guessed:

```
export const WARNABLE = new Set<EventType>([
  EventType.FULLSCREEN_EXIT, // the Fullscreen API said so - not an inference
  EventType.TAB_HIDDEN,
  EventType.SCREEN_SHARE_STOPPED,
  EventType.DEVTOOLS_SUSPECTED,
]);
```

No ML detection is in this set. **PHONE_DETECTED**, **MULTIPLE_FACES**, **FACE_MISMATCH**, **SPEECH_DETECTED** and friends never reach this handler at all — they take the human-review path in §7.5 and only become warnable after an admin approves them. That separation is the core safety property of the whole system: a model can raise a hand, but only a person can raise a warning.

6.4 Failure modes to handle before exam day

- Alt-Tab on Windows fires `blur` but not `fullscreenchange` — that's why you need both listeners.
- OS notifications and update popups steal focus. The 2-second blur grace absorbs most.
- `requestFullscreen()` rejects without a gesture — always route through a button.
- iOS Safari has no Fullscreen API on iPhone → block mobile entirely at preflight.
- A student with a genuinely dying laptop will trip 3 warnings in 10 minutes. Blocking must be **admin-reversible** in one click, never final.

6.5 The hard-stop path — screen violations never go to review

The camera and audio detectors go to a human (\$7.5) because a model is *guessing*. Screen and focus violations are different: the browser **observed a fact**. The Fullscreen API did not infer that fullscreen ended — it reported it. `visibilitychange` is not a probability. There is nothing for an admin to adjudicate, and making a proctor click "approve" on 400 tab-switches would bury the queue that actually matters.

So these take a **fully automatic path**: flag → flag → flag → **auto-submit and log out**.

```
export const HARD_STOP = new Set<EventType>([
  EventType.FULLSCREEN_EXIT,      // left the exam window
  EventType.TAB_HIDDEN,          // switched tab or minimised
  EventType.SCREEN_SHARE_STOPPED, // killed the screen share
  EventType.DEVTOOLS_SUSPECTED,  // opened developer tools
]);
export const HARD_STOP_LIMIT = 3; // Exam.maxWarnings
```

6.5.1 Flag 1 and 2 — warn, pause, capture

Identical to §6.3: the exam locks behind an overlay, evidence is captured, the counter increments server-side, and the student must click to return to fullscreen. The overlay states the count plainly: "Flag 2 of 3. One more and your test will be submitted automatically." No ambiguity — a student who is about to lose their exam deserves to know it.

6.5.2 Flag 3 — terminate

```
// apps/api/src/proctor/hard-stop.service.ts
async applyHardStop(attemptId: string, triggeringEventId: string) {
  return this.db.transaction(async tx => {
    const attempt = await tx.attempt.findUniqueOrThrow({
      where: { id: attemptId }, include: { student: true, exam: true },
    });
    if (attempt.status !== 'IN_PROGRESS') return; // idempotent - never double-fire

    // 1. SUBMIT, don't discard. Everything they answered still counts.
    await tx.attempt.update({
      where: { id: attemptId },
      data: {
        status: 'BLOCKED',
        submittedAt: new Date(),
        blockedReason: `Auto-submitted: ${HARD_STOP_LIMIT} screen violations`,
        blockedAt: new Date(),
        blockedByEventId: triggeringEventId,
      },
    });

    // 2. Freeze the evidence into a case file before anything can change
```

```

await this.evidence.bundle(attemptId, triggeringEventId);

// 3. Revoke the session - this is the "logged out" half
await this.sessions.revoke(attempt.studentId, 'HARD_STOP');

// 4. Tell the client to tear itself down
this.push(attempt.student.rollNumber, 'exam:terminated', {
  reason: 'SCREEN_VIOLATION_LIMIT',
  flags: HARD_STOP_LIMIT,
  submittedAt: new Date().toISOString(),
  message: 'Your test has been submitted automatically and you have been signed out. '
    + 'Your answers were saved. If you believe this was an error, contact the exam office.',
  appealable: true,
});

// 5. Surface to the admins - not for approval, for awareness
this.push('admins', 'student:terminated', {
  rollNumber: attempt.student.rollNumber, name: attempt.student.name,
  answered: await this.countAnswers(attemptId), riskScore: attempt.riskScore,
  unblockable: true,
});

await this.audit.log('SYSTEM', 'hard-stop', 'AUTO_SUBMIT_AND_LOGOUT',
  attemptId, { triggeringEventId });
});
}

```

Logging out properly matters more than it looks. A JWT is stateless — deleting a session row does not stop a token that's already in the student's memory. You need all three:

```

// apps/api/src/auth/sessions.service.ts
async revoke(studentId: string, reason: string) {
  const session = await this.redis.get(`session:${studentId}`); // Upstash on Vercel
  if (session) {
    const { jti } = JSON.parse(session);
    // a) deny the access token for the remainder of its 15-minute life
    await this.redis.set(`denied:${jti}`, reason, 'EX', 900);
  }
  await this.redis.del(`session:${studentId}`); // b) kill the session record
  await this.db.refreshToken.deleteMany({ where: { studentId } }); // c) kill refresh rotation
  this.disconnectSockets(studentId, reason);
}

```

Your auth guard must check the denylist on every request, or a terminated student keeps working for up to 15 minutes on a token they already hold.

On Supabase: `supabase.auth.admin.signOut(userId, 'global')` revokes refresh tokens, but an already-issued access token stays valid until it expires. Keep the `denied:{jti}` check in an Edge Function middleware, or shorten the access-token TTL to ~5 minutes so the window closes fast.

Also add an RLS predicate so a blocked attempt cannot be written to at all — belt and braces:

```

create policy "no writes to a blocked attempt" on answers
for insert to authenticated
with check (
  exists (select 1 from attempts a
    where a.id = attempt_id
      and a.student_id = auth.uid()
      and a.status = 'IN_PROGRESS')
);

```

That policy is the real enforcement. Even a student running a patched client with a stolen token writes nothing.

6.5.3 Client teardown

The client must not merely navigate — it must release the hardware and destroy its credentials, or the camera light stays on and the token lingers:

```

socket.on('exam:terminated', async payload => {
  proctorAgent.stop(); // detach every listener
  [webcamStream, screenStream, micStream].forEach(s =>
    s?.getTracks().forEach(t => t.stop()); // camera/mic light OFF
  mediaRecorder?.stop();
  await flushOfflineQueue(); // last evidence out of IndexedDB
  authStore.clear(); // in-memory token gone
  if (document.fullscreenElement) await document.exitFullscreen();
  router.replace(`/exam/terminated?r=${payload.reason}`); // terminal route, no back
});

```

The terminated page shows what was submitted, how many questions were answered, the three flags with timestamps, and **how to appeal**. A student staring at a dead screen with no explanation is how a technical event becomes a complaint.

6.5.4 The three guards that keep this fair

Automatic termination is severe, so three protections are mandatory, not optional:

1. **Answers are submitted, never discarded.** Someone terminated at question 48 of 60 is graded on 48 questions. Termination ends the attempt; it does not void the work.
2. **The 60-second grace window** (§17.2 #17). The first flag inside the first minute is a tutorial, not a flag — that is when honest students fumble the fullscreen prompt.
3. **Every termination is reversible by an admin** (§13.4). This is not a courtesy; it's the release valve that makes an automatic rule acceptable.

7. Feature 4 — Camera Proctoring with YOLO

7.1 Design decision: where does inference run?

Option	Latency	Server cost	Quality	Verdict
Client-side (ONNX Runtime Web / TF.js, YOLO11n WASM or WebGPU)	instant	~zero	good, but tamperable	fast pre-filter
Server-side (FastAPI + Ultralytics on GPU)	0.5–3 s	needs 1 GPU	best, trustworthy	the authority

Recommended hybrid. The browser runs a tiny YOLO11n every 3 s only to answer *"is this frame interesting?"*. Interesting frames — plus one baseline frame every 30 s regardless — upload to the server, where the real model decides. This cuts upload volume roughly 10× while keeping the trust boundary on your side. **Never let the client's verdict alone create a violation.**

7.2 Client capture loop

```
// apps/web/src/proctor/webcam.ts
export class WebcamProctor {
  private video = document.createElement('video');
  private canvas = new OffscreenCanvas(640, 480);
  private detector?: YoloWeb;

  async start(stream: MediaStream) {
    this.video.srcObject = stream;
    await this.video.play();
    this.detector = await YoloWeb.load('/models/yolo11n.onnx'); // ~6 MB, HTTP-cached

    stream.getVideoTracks()[0]
      .addEventListener('ended', () => this.report('CAMERA_STREAM_LOST', 'CRITICAL'));

    setInterval(() => this.tick(), 3000);
    setInterval(() => this.upload('PERIODIC'), 30000);
  }

  private async tick() {
    const bmp = await createImageBitmap(this.video);
    this.canvas.getContext('2d')!.drawImage(bmp, 0, 0, 640, 480);

    if (await meanLuma(this.canvas) < 12) // covered lens / black frame
      return this.report('CAMERA_BLOCKED', 'HIGH');

    const dets = await this.detector!.run(this.canvas); // [{cls, conf, box}]
    const persons = dets.filter(d => d.cls === 'person' && d.conf > 0.50);
    const phones = dets.filter(d => d.cls === 'cell phone' && d.conf > 0.35);
    const books = dets.filter(d => ['book', 'laptop', 'tv', 'remote'].includes(d.cls) && d.conf > 0.45);

    if (persons.length !== 1 || phones.length || books.length)
      await this.upload('SUSPECT', { persons: persons.length, phones, books });
  }

  private async upload(reason: string, hint?: any) {
    const blob = await this.canvas.convertToBlob({ type: 'image/jpeg', quality: 0.6 }); // ~35 KB
    const fd = new FormData();
    fd.append('frame', blob);
    fd.append('attemptId', this.attemptId);
    fd.append('reason', reason);
    fd.append('hint', JSON.stringify(hint ?? {}));
    navigator.sendBeacon('/api/proctor/webcam-frame', fd) ||
      fetch('/api/proctor/webcam-frame', { method: 'POST', body: fd, keepalive: true });
  }
}
```

7.3 Server-side ML service

```
# apps/ml/main.py
from fastapi import FastAPI, UploadFile, Form
from ultralytics import YOLO
import insightface, numpy as np, cv2

app = FastAPI()
det = YOLO("yolo11n.pt") # yolo11s.pt if you have GPU headroom
face = insightface.app.FaceAnalysis(name="buffalo_l"); face.prepare(ctx_id=0)

SUSPECT = {"cell phone": 0.30, "book": 0.45, "laptop": 0.45, "tv": 0.45, "remote": 0.45}

@app.post("/analyze/frame")
async def analyze(frame: UploadFile, roll_number: str = Form(...)):
    img = cv2.imdecode(np.frombuffer(await frame.read(), np.uint8), cv2.IMREAD_COLOR)

    r = det.predict(img, imgsz=640, conf=0.25, verbose=False)[0]
    objs = [{"cls": det.names[int(b.cls)], "conf": float(b.conf),
            "box": [float(x) for x in b.xyxy[0]]} for b in r.bboxes]

    persons = [o for o in objs if o["cls"] == "person" and o["conf"] > 0.50]
    phones = [o for o in objs if o["cls"] == "cell phone" and o["conf"] > SUSPECT["cell phone"]]
    others = [o for o in objs if o["cls"] in SUSPECT and o["cls"] != "cell phone"
            and o["conf"] > SUSPECT[o["cls"]]]

    faces, identity = face.get(img), None
    if len(faces) == 1:
        emb = faces[0].normed_embedding
        identity = {"similarity": float(cosine(emb, load_embedding(roll_number))),
                  "yaw_proxy": yaw_from_kps(faces[0].kps)} # crude "looking away" signal

    events = []
    if len(persons) == 0: events.append(("NO_FACE", "MEDIUM"))
    if len(persons) > 1: events.append(("MULTIPLE_FACES", "CRITICAL"))
    if phones: events.append(("PHONE_DETECTED", "CRITICAL"))
    if others: events.append(("BOOK_DETECTED", "HIGH"))
    if identity and identity["similarity"] < 0.35: events.append(("FACE_MISMATCH", "CRITICAL"))
    if identity and abs(identity["yaw_proxy"]) > 0.45: events.append(("LOOKING_AWAY", "LOW"))

    return {"objects": objs, "faces": len(faces), "identity": identity, "events": events}
```

7.4 Temporal smoothing — the first filter

Never raise anything from one frame. Require N of the last M frames to agree, then cool down. This is a **noise filter**, not a **decision** — its only job is to decide whether a detection is stable enough to be worth a human's attention.

```
// apps/worker/src/proctor/smoothing.ts
const RULES = {
  PHONE_DETECTED: { n: 2, m: 4, cooldownSec: 60 }, // ~2 of 4 frames within 12s
  MULTIPLE_FACES: { n: 3, m: 5, cooldownSec: 60 },
  NO_FACE: { n: 5, m: 6, cooldownSec: 90 }, // looking down at scratch paper - be lenient
  LOOKING_AWAY: { n: 8, m: 10, cooldownSec: 120 },
  FACE_MISMATCH: { n: 4, m: 5, cooldownSec: 300 },
};

async function confirm(attemptId: string, type: keyof typeof RULES) {
  const key = `win:${attemptId}:${type}`;
  await redis.lpush(key, Date.now());
  await redis.ltrim(key, 0, RULES[type].m - 1);
  await redis.expire(key, 300);

  if ((await redis.llen(key)) < RULES[type].n) return false;
  // cooldown so one phone lying on the desk isn't 40 separate review items
  return (await redis.set(`cool:${attemptId}:${type}`, '1', 'EX', RULES[type].cooldownSec, 'NX')) !== null;
}
```

A detection that survives this filter does **not** become a violation. It becomes a **review item**, which is \$7.5.

7.5 Human-in-the-loop adjudication — the ML never flags anyone by itself

This is the design rule that makes the whole camera/audio layer defensible:

A model may only propose. Only an admin may flag.

Nothing YOLO, InsightFace or the VAD produces ever touches `warningCount`, `riskScore`, or the student's exam state on its own. It produces a **proposal** with evidence attached, an admin looks at the two snapshots, and only an **Approve** click converts it into a real violation.

7.5.1 The pipeline



Untouched proposals **expire harmlessly**: a sweep marks anything older than 15 minutes **EXPIRED**, weight 0. A busy proctor bench can never accidentally punish someone through inaction, and it can never accidentally punish someone through a backlog either.

7.5.2 Creating the proposal (never a violation)

```
// apps/worker/src/proctor/ml-proposal.processor.ts
@Processor('analyze-frame')
export class MLProposalProcessor {
  async process(job: Job<{ attemptId: string; assetId: string }>) {
    const { attemptId, assetId } = job.data;
    const asset = await this.prisma.mediaAsset.findUniqueOrThrow({ where: { id: assetId } });
    const result = await this.ml.analyzeFrame(asset, attemptId); // FastAPI, §7.3

    for (const [type, severity] of result.events) {
      // filter 1: does it survive N-of-M + cooldown?
      if (!(await confirm(attemptId, type))) continue;

      // filter 2: is anything still pending for this attempt+type? don't stack duplicates
      const open = await this.prisma.proctorEvent.findFirst({
        where: { attemptId, type, status: 'PENDING_REVIEW' },
      });
      if (open) {
        // attach the new frame to the existing item instead of creating a second one
        await this.attachFrame(open.id, assetId);
        continue;
      }
    }
  }
}
```

```

const ev = await this.prisma.proctorEvent.create({
  data: {
    attemptId, type, severity,
    source: 'ML_PROPOSAL',
    status: 'PENDING_REVIEW',
    weight: WEIGHTS[type], // the weight it WOULD carry if approved
    appliedWeight: 0, // <-- what actually counts today: nothing
    confidence: result.confidenceFor(type),
    payload: { objects: result.objects, identity: result.identity },
    clientTs: asset.capturedAt,
    webcamShot: assetId,
  },
});

// pull a matching screen snapshot + a short frame strip for context
await this.evidence.assembleForReview(ev.id, {
  screenShotNear: asset.capturedAt, // nearest stored SCREEN_FRAME
  stripSeconds: 6, // ±3s of webcam frames around the hit
});

this.gateway.server.to('admins').emit('review:new', await this.reviewCard(ev.id));
}
}
}

```

Note what is **absent**: no `attempt.update`, no `warningCount`, no `riskScore`, no emit to the student. The student's exam is completely unaffected at this point, and they are shown nothing.

7.5.3 The admin decision endpoint

```

// apps/api/src/proctor/review.controller.ts
@Post('review/:eventId/decide')
@Roles('PROCTOR', 'SUPER_ADMIN')
async decide(@Param('eventId') eventId: string, @Body() dto: DecideDto, @Req() req) {
  const admin = req.user;

  // atomic: two of the five stations can't adjudicate the same item
  const claimed = await this.redis.set(`review:${eventId}`, admin.id, 'EX', 120, 'NX');
  if (!claimed) throw new ConflictException('Another proctor is reviewing this item.');
```

```

  return this.prisma.$transaction(async tx => {
    const ev = await tx.proctorEvent.findUniqueOrThrow({ where: { id: eventId } });
    if (ev.status !== 'PENDING_REVIEW')
      throw new ConflictException('Already ${ev.status}');

    // ----- DISMISS -----
    if (dto.verdict === 'FALSE_POSITIVE' || dto.verdict === 'INCONCLUSIVE') {
      await tx.proctorEvent.update({
        where: { id: eventId },
        data: { status: 'DISMISSED', verdict: dto.verdict, appliedWeight: 0,
          reviewerId: admin.id, reviewedAt: new Date(), reviewNote: dto.note },
      });
      await this.metrics.recordFalsePositive(ev.type); // tunes thresholds later, §7.5.6
      await this.audit.log('ADMIN', admin.id, 'REVIEW_DISMISS', eventId, { type: ev.type });
      return { ok: true, applied: false }; // student is never told
    }

    // ----- APPROVE -----
    const attempt = await tx.attempt.findUniqueOrThrow({

```

```

    where: { id: ev.attemptId }, include: { exam: true, student: true },
  });

  // the admin chooses the consequence - approving evidence is not the same as punishing
  const action = dto.action as 'RECORD_ONLY' | 'WARN' | 'BLOCK';

  await tx.proctorEvent.update({
    where: { id: eventId },
    data: { status: 'CONFIRMED', verdict: 'CHEATING', appliedWeight: ev.weight,
      reviewerId: admin.id, reviewedAt: new Date(), reviewNote: dto.note },
  });

  const warningCount = attempt.warningCount + (action === 'WARN' || action === 'BLOCK' ? 1 : 0);
  const blocked = action === 'BLOCK' || warningCount >= attempt.exam.maxWarnings;

  await tx.attempt.update({
    where: { id: attempt.id },
    data: {
      riskScore: attempt.riskScore + ev.weight,
      warningCount,
      ...(blocked && { status: 'BLOCKED',
        blockedReason: `${ev.type} confirmed by proctor ${admin.name}` }),
    },
  });

  await this.audit.log('ADMIN', admin.id, `REVIEW_APPROVE_${action}`, eventId,
    { rollNumber: attempt.student.rollNumber, type: ev.type });

  // NOW the student hears about it - with a human name behind it
  this.gateway.server.to(`student:${attempt.student.rollNumber}`).emit(
    blocked ? 'blocked' : 'warning',
    { warningCount, maxWarnings: attempt.exam.maxWarnings,
      reason: HUMAN_READABLE[ev.type], reviewedBy: admin.name,
      appealable: true },
  );

  if (blocked) await this.evidence.bundle(attempt.id, eventId);
  return { ok: true, applied: true, blocked };
}

```

Two-person rule for blocks. Wire `BLOCK` on a `CRITICAL` item to require a second admin's confirm before it commits — §17.3 #24. One panicked proctor should not be able to end thirty exams.

7.5.4 The review card the admin actually sees

The whole point is that the decision takes **under five seconds**. Everything needed is on one card, nothing requires a click-through:

```

// apps/web/src/app/admin/review-queue/ReviewCard.tsx
export function ReviewCard({ item, onDecide }: Props) {
  useHotkeys('a', () => onDecide('APPROVE', 'WARN')); // keyboard-first: 450 students, 5 people
  useHotkeys('d', () => onDecide('DISMISS'));
  useHotkeys('r', () => onDecide('APPROVE', 'RECORD_ONLY'));

  return (
    <article className="rounded-lg border-2 border-amber-400 bg-amber-50/40 p-3">
      <header className="flex items-center justify-between">
        <div>

```

```

        <span className="font-mono font-bold">{item.rollNumber}</span>
        <span className="ml-2 text-sm text-gray-600">{item.name}</span>
    </div>
    <div className="text-right">
        <Badge severity={item.severity}>{HUMAN_READABLE[item.type]}</Badge>
        <div className="text-xs text-gray-500">
            confidence {(item.confidence * 100).toFixed(0)}% {timeAgo(item.serverTs)}
        </div>
    </div>
</header>

{/* the two snapshots, side by side - this is the evidence */}
<div className="mt-2 grid grid-cols-2 gap-2">
    <figure>
        <img src={item.webcamUrl} className="w-full rounded"
            /* server-drawn bounding box on the proposed object */ />
        <figcaption className="text-[11px] text-gray-500">
            Webcam box = {item.payload.objects?.[0]?.cls}
        </figcaption>
    </figure>
    <figure>
        <img src={item.screenUrl} className="w-full rounded" />
        <figcaption className="text-[11px] text-gray-500">
            Their screen at the same moment
        </figcaption>
    </figure>
</div>

{/* 3s of webcam frames - kills "it was a water bottle" instantly */}
<FrameStrip frames={item.strip} className="mt-2" />

{/* context that changes the read */}
<dl className="mt-2 flex gap-4 text-xs text-gray-600">
    <div>risk <b>{item.riskScore}</b></div>
    <div>warnings <b>{item.warningCount}</b></div>
    <div>prior dismissed <b>{item.priorDismissed}</b></div>
    <div>Q{item.currentQuestion}</div>
</dl>

<footer className="mt-3 flex gap-2">
    <Button variant="destructive" onClick={() => onDecide('APPROVE', 'WARN')}>
        Approve Warn <kbd>A</kbd>
    </Button>
    <Button variant="outline" onClick={() => onDecide('APPROVE', 'RECORD_ONLY')}>
        Approve Record only <kbd>R</kbd>
    </Button>
    <Button variant="ghost" onClick={() => onDecide('DISMISS')}>
        Dismiss <kbd>D</kbd>
    </Button>
    <Button variant="ghost" onClick={() => openLiveView(item.rollNumber)}>
        Watch live
    </Button>
    <Button variant="ghost" onClick={() => openChat(item.rollNumber)}>
        Ask the student
    </Button>
</footer>
</article>
);
}

```

Two options matter more than they look:

- **"Approve → Record only"** — the proctor agrees a phone is visible but judges it not worth a warning yet. The event is confirmed and counts toward `riskScore` and the evidence file, but the student is never interrupted. Most first offences should land here.
- **"Ask the student"** — opens the §12 chat. "Please move the object on your desk out of frame." This resolves a large share of items without any punishment at all, and it is the single most humane feature in the system.

7.5.5 Keeping the queue survivable for 5 people

450 students × ~90 minutes will generate real volume. Without throttling, the queue becomes wallpaper and the humans start rubber-stamping — which silently returns you to auto-flagging, with worse accuracy.

Control	Rule
Per-student cap	Max 3 pending items per student at once. Further detections attach frames to the existing item instead of creating new rows.
Global admission control	If the queue exceeds ~40 items, raise the confidence floor dynamically (only <code>conf > 0.55</code> phones get in) and log the drop rate. Never let the queue outrun the humans.
Priority ordering	<code>severity DESC, confidence DESC, riskScore DESC, age ASC . PHONE_DETECTED</code> at 0.9 on a student already at risk 70 outranks a <code>LOOKING_AWAY</code> at 0.4.
Auto-expiry	15 minutes unreviewed → <code>EXPIRED</code> , weight 0, counted in metrics as "capacity missed".
Claim lock	Redis <code>SET NX</code> , 120 s TTL — the same mechanism as chat threads (§12.3), so two stations never adjudicate one item.
Suppress after dismissal	Dismissing <code>PHONE_DETECTED</code> for a student suppresses that type for them for 10 minutes. If the water bottle is a water bottle, it stays a water bottle.
Batch review	Same student, same type, 3+ items → one grouped card with a frame grid, one decision.

```
// dynamic admission control - protects the humans from the model
async function admissionThreshold(): Promise<number> {
  const depth = await redis.zcard('review:queue');
  if (depth < 15) return 0.35;      // plenty of capacity, let borderline items through
  if (depth < 40) return 0.50;
  if (depth < 80) return 0.65;
  return 0.80;                    // triage mode: only near-certain detections
}
```

7.5.6 The feedback loop — this is why the design pays off twice

Every dismissal is a labelled false positive, tied to a stored frame. That gives you something no auto-flagging system ever gets:

```
-- per-detector precision, live, during the exam
SELECT type,
  COUNT(*) FILTER (WHERE status='CONFIRMED') AS approved,
  COUNT(*) FILTER (WHERE status='DISMISSED') AS dismissed,
  ROUND(100.0 * COUNT(*) FILTER (WHERE status='CONFIRMED')
    / NULLIF(COUNT(*) FILTER (WHERE status IN ('CONFIRMED', 'DISMISSED')),0), 1) AS precision_pct
FROM "ProctorEvent"
WHERE source='ML_PROPOSAL' AND "attemptId" IN (SELECT id FROM "Attempt" WHERE "examId"=§1)
GROUP BY type ORDER BY dismissed DESC;
```

Put this on the admin dashboard as a live tile. If `PHONE_DETECTED` shows 12% precision an hour in, you raise its confidence floor **during the exam** — and because nothing was auto-flagged, no student was harmed by the bad threshold. Export the dismissed frames afterwards and you have a purpose-built fine-tuning set for the next exam.

7.5.7 What this costs and what it buys

Cost: a proctor decision is roughly 5 seconds. At a realistic ~ 1.5 reviewable proposals per student across a 90-minute exam, that's ~ 675 items $\div 5$ proctors ≈ 135 each $\approx$ **11 minutes of decision time per proctor**, spread over 90 minutes. Entirely feasible — *provided* §7.5.5's throttles are actually implemented. Skip them and the queue hits several thousand and the system fails.

Buys: - Zero students punished by a model error. - Every sanction has a named human behind it — which is exactly what you need when a student appeals, and what §16.6 legally requires. - Live precision metrics and a free labelled dataset. - You can run detectors *aggressively* (low confidence floors, more classes) without risk, because the human is the gate. Auto-flagging forces you to run them conservatively, so you catch **less**.

That last point is the counter-intuitive one: **human-in-the-loop lets you detect more, not less**.

7.6 Capacity math — do this before you buy hardware

- $450 \text{ students} \times 1 \text{ frame} / 3 \text{ s} = 150 \text{ fps}$ if everything went server-side. That needs a serious GPU.
 - With the client pre-filter: $\sim 10\text{--}15\%$ "interesting" + 1 baseline / 30 s $\rightarrow 450/30 + 0.12 \times 150 \approx 33 \text{ fps}$.
 - YOLO11n at 640 px on an RTX 3060 / T4 runs $\sim 120\text{--}200$ fps batched. One GPU is comfortable. Batch 8–16 frames per call.
 - Storage: $35 \text{ KB/frame} \times \sim 120 \text{ stored frames/student} \times 450 \approx 1.9 \text{ GB}$ per exam. Trivial.
 - CPU-only fallback: YOLO11n ONNX on 8 cores $\approx 25\text{--}35$ fps. Tight but survivable if you stretch the baseline interval to 60 s.
-

8. Feature 5 — Screen Capture / Periodic Screenshots

8.1 What is actually possible in a browser

You **cannot** silently screenshot a user's screen. The only API is `getDisplayMedia()`, which requires an explicit permission prompt and shows an OS/browser sharing indicator. That's a hard constraint — design around it.

What you *can* do: - Require, at preflight, that the student shares **the entire screen** — and verify it. - Once granted, capture frames from that stream silently and continuously for the whole exam. - Detect and violate if they stop sharing or switch to a single-window share.

8.2 Enforcing "entire screen"

```
const stream = await navigator.mediaDevices.getDisplayMedia({
  video: { displaySurface: 'monitor', frameRate: 2 },
  audio: false,
  // @ts-expect-error Chrome-only hints that hide the tab/window options in the picker
  monitorTypeSurfaces: 'include', selfBrowserSurface: 'exclude', surfaceSwitching: 'exclude',
});

const s = stream.getVideoTracks()[0].getSettings();
if (s.displaySurface !== 'monitor') {
  stream.getTracks().forEach(t => t.stop());
  throw new PreflightError('You must share your ENTIRE SCREEN, not a tab or a window.');
```

```
stream.getVideoTracks()[0].addEventListener('ended',
  () => agent.report({ type: 'SCREEN_SHARE_STOPPED', severity: 'CRITICAL' }));
```

Re-verify `displaySurface` on every heartbeat — a student can switch surfaces mid-exam from the browser's share bar.

8.3 Periodic screenshots (recommended default)

```
// apps/web/src/proctor/screen.ts
export class ScreenProctor {
  private track!: MediaStreamTrack;
  private capture!: ImageCapture;

  async start(stream: MediaStream) {
    this.track = stream.getVideoTracks()[0];
    this.capture = new ImageCapture(this.track);
    setInterval(() => this.shoot('PERIODIC'), 20000); // every 20s
  }

  async shoot(reason: string): Promise<Blob> {
    const bmp = await this.capture.grabFrame();
    const c = new OffscreenCanvas(1280, Math.round(1280 * bmp.height / bmp.width));
    c.getContext('2d')!.drawImage(bmp, 0, 0, c.width, c.height);
    const blob = await c.convertToBlob({ type: 'image/webp', quality: 0.5 }); // ~60 KB
    await this.upload(blob, reason);
    return blob;
  }
}
```

At 20-second intervals over a 90-minute exam: 270 frames × 60 KB × 450 students ≈ 7.3 GB. Fine on local MinIO; budget for it on S3.

8.4 Full screen recording (optional, much heavier)

```
const rec = new MediaRecorder(stream, {
  mimeType: 'video/webm;codecs=vp9',
  videoBitsPerSecond: 250_000, // ~1.9 MB/min
});
let seq = 0;
rec.ondataavailable = e => {
  if (!e.data.size) return;
  const fd = new FormData();
  fd.append('chunk', e.data);
  fd.append('attemptId', attemptId);
  fd.append('seq', String(seq++));
  fetch('/api/proctor/screen-chunk', { method: 'POST', body: fd, keepalive: true });
};
rec.start(10_000); // a 10-second chunk - resumable, survives a crash
```

A worker stitches chunks with `ffmpeg -f concat` at exam end.

Budget: $1.9 \text{ MB/min} \times 90 \text{ min} \times 450 = \sim 77 \text{ GB per exam}$. Recommendation: record only students whose `riskScore` crosses a threshold, and flip recording on remotely via a socket command (`cmd:record`).

8.5 Storage layout & retention

```
s3://lead-exams/
  {examId}/
    {rollNumber}/
      webcam/2026-09-10T10-32-11.123Z.jpg
      screen/2026-09-10T10-32-20.000Z.webp
      audio/2026-09-10T10-32-00.000Z.opus
      evidence/{eventId}/{webcam.jpg, screen.webp, meta.json}
      recording/session.webm
      manifest.json          # sha256 of every asset - tamper-evident chain
```

- Buckets are **private**. Access only via short-lived (5 min) presigned URLs, generated after a role check plus an `AuditLog` write.
- Lifecycle: delete non-flagged media after **30 days**; keep flagged evidence **1 year** or until appeals close. Put these numbers in the consent form.

9. Feature 6 — Microphone / Audio Monitoring

9.1 Two-tier approach

Tier 1 — client, always on, zero bandwidth. Compute RMS and zero-crossing rate in an `AudioWorklet`. Report *metrics* only; never audio, unless something fires.

```
// apps/web/public/worklets/rms.js
class RmsProcessor extends AudioWorkletProcessor {
  process(inputs) {
    const ch = inputs[0][0];
    if (!ch) return true;
    let sum = 0, zc = 0;
    for (let i = 0; i < ch.length; i++) {
      sum += ch[i] * ch[i];
      if (i && (ch[i] >= 0) !== (ch[i - 1] >= 0)) zc++;
    }
    this.port.postMessage({ rms: Math.sqrt(sum / ch.length), zcr: zc / ch.length });
    return true;
  }
}
registerProcessor('rms-processor', RmsProcessor);
```

```
// main thread
node.port.onmessage = ({ data }) => {
  const db = 20 * Math.log10(data.rms + 1e-9);
  const speechLike = db > baselineDb + 18 && data.zcr > 0.05 && data.zcr < 0.25;

  if (speechLike) {
    if (++speechFrames > 12) { // ~1.2s of sustained speech
      agent.report({ type: 'SPEECH_DETECTED', severity: 'MEDIUM', payload: { db, zcr: data.zcr } });
      audioProctor.uploadClip(10); // last 10s from the rolling buffer
      speechFrames = 0;
    }
  } else speechFrames = Math.max(0, speechFrames - 1);
};
```

Calibrate `baselineDb` during preflight: ask for 5 seconds of silence and take the median.

Tier 2 — server, on trigger. Keep a rolling 30-second Opus ring buffer; on a trigger, upload the last 10 s.

```
# apps/ml/routers/audio.py
import webrtcvad
vad = webrtcvad.Vad(2)

@router.post("/analyze/audio")
async def analyze_audio(clip: UploadFile):
    pcm = to_pcm16_mono_16k(await clip.read())
    frames = [pcm[i:i+480] for i in range(0, len(pcm) - 480, 480)] # 30ms frames
    voiced = sum(vad.is_speech(f.tobytes(), 16000) for f in frames)
    ratio = voiced / max(len(frames), 1)

    speakers = estimate_speakers(pcm) if ratio > 0.30 else 1 # pyannote or MFCC clustering

    ev = []
    if ratio > 0.35: ev.append(("SPEECH_DETECTED", "MEDIUM"))
    if speakers > 1: ev.append(("MULTIPLE_VOICES", "HIGH"))
    return {"voiced_ratio": ratio, "speakers": speakers, "events": ev}
```

9.2 Honest limits

- **Do not transcribe.** Running ASR on 450 students' rooms is a privacy and legal problem, and it will capture family conversations. Detect *that* speech happened; let a human listen to the 10-second clip.

- Audio is the noisiest signal you have. Weight it **low** (`SPEECH_DETECTED` weight 3, never warnable). Use it to *rank* who a proctor should look at — never to auto-block.
- State clearly in the consent form that audio is recorded on trigger.

10. Feature 7 — Anomaly Event Pipeline

Your requirement: on any suspicious anomaly, capture a screen screenshot **and** a webcam snapshot, bind them to the event, store them under the student's roll number.

10.1 Client — synchronous dual capture

```
// apps/web/src/proctor/evidence.ts
export async function captureEvidence(reason: EventType, extra?: any) {
  const clientEventId = crypto.randomUUID(); // correlates every piece
  const ts = Date.now();

  const [webcam, screen] = await Promise.allSettled([
    webcamProctor.grabBlob(), // ~35 KB jpeg
    screenProctor.shoot(reason), // ~60 KB webp
  ]);

  const fd = new FormData();
  fd.append('attemptId', attemptId);
  fd.append('clientEventId', clientEventId);
  fd.append('reason', reason);
  fd.append('clientTs', String(ts));
  fd.append('meta', JSON.stringify({
    url: location.pathname,
    fs: !!document.fullscreenElement,
    hidden: document.hidden,
    screens: (window as any).screen?.isExtended,
    currentQuestion: examStore.index,
    ...extra,
  }));
  if (webcam.status === 'fulfilled') fd.append('webcam', webcam.value, 'webcam.jpg');
  if (screen.status === 'fulfilled') fd.append('screen', screen.value, 'screen.webp');

  // keepalive so it survives a tab close or navigation
  await fetch('/api/proctor/evidence', { method: 'POST', body: fd, keepalive: true })
    .catch(() => idb.queue('evidence', fd)); // retry on reconnect
  return clientEventId;
}
```

Call it on: `FULLSCREEN_EXIT`, `TAB_HIDDEN`, `WINDOW_BLUR`, `PRINTSCREEN_KEY`, `DEVTOOLS_SUSPECTED`, `PHONE_DETECTED`, `MULTIPLE_FACES`, `FACE_MISMATCH`, `SPEECH_DETECTED`, `MULTIPLE_VOICES`, `SCREEN_SHARE_STOPPED`, and any admin-initiated "capture now".

10.2 Server — ingest, store, notify

```
// apps/api/src/proctor/evidence.controller.ts
@Post('evidence')
@UseInterceptors(FileFieldsInterceptor(
  [{ name: 'webcam', maxCount: 1 }, { name: 'screen', maxCount: 1 }],
  { limits: { fileSize: 3 * 1024 * 1024 } })
async ingest(@UploadedFiles() files, @Body() dto: EvidenceDto, @Req() req) {
  const attempt = await this.svc.assertOwned(req.user.studentId, dto.attemptId);
  const roll = attempt.student.rollNumber;

  const assets = [];
  for (const [kind, ext, f] of [
    ['WEBCAM_FRAME', 'jpg', files.webcam?.[0]],
    ['SCREEN_FRAME', 'webp', files.screen?.[0]],
  ] as const) {
    if (!f) continue;
    const key = `${attempt.examId}/${roll}/evidence/${dto.clientEventId}/${kind}.${ext}`;
    const sha = createHash('sha256').update(f.buffer).digest('hex');
    await this.s3.put(key, f.buffer, f.mimetype);
    assets.push(await this.prisma.mediaAsset.create({
      data: { attemptId: attempt.id, kind, objectKey: key, sha256: sha,
        bytes: f.size, capturedAt: new Date(+dto.clientTs) },
    }));
  }

  const ev = await this.prisma.proctorEvent.create({
    data: {
      attemptId: attempt.id, type: dto.reason, severity: SEVERITY[dto.reason],
      weight: WEIGHTS[dto.reason] ?? 0, payload: JSON.parse(dto.meta),
      clientTs: new Date(+dto.clientTs),
      webcamShot: assets.find(a => a.kind === 'WEBCAM_FRAME')?.id,
      screenShot: assets.find(a => a.kind === 'SCREEN_FRAME')?.id,
    },
  });

  await this.queue.add('analyze-evidence', { eventId: ev.id }); // YOLO on the webcam shot
  this.gateway.server.to('admins').emit('live:alert', {
    rollNumber: roll, eventId: ev.id, type: dto.reason,
    severity: SEVERITY[dto.reason], thumbs: assets.map(a => a.id),
  });
  return { ok: true, eventId: ev.id };
}
```

10.3 Risk scoring — turn 200 noisy events into one number

```
export const WEIGHTS: Record<EventType, number> = {
  PHONE_DETECTED: 40, MULTIPLE_FACES: 40, FACE_MISMATCH: 35,
  DEVTOOLS_SUSPECTED: 30, SCREEN_SHARE_STOPPED: 25, CAMERA_BLOCKED: 25,
  IMPOSSIBLE_TYPING_SPEED: 25, RAPID_ANSWER_PATTERN: 20, MULTIPLE_VOICES: 20,
  FULLSCREEN_EXIT: 15, TAB_HIDDEN: 15, PRINTSCREEN_KEY: 15,
  BOOK_DETECTED: 12, IP_CHANGED: 10, WINDOW_BLUR: 8, PASTE_ATTEMPT: 8,
  NO_FACE: 6, COPY_ATTEMPT: 5, SPEECH_DETECTED: 3, LOOKING_AWAY: 2,
  CONTEXT_MENU: 1, SHORTCUT_BLOCKED: 1,
};
```

A weight only lands if the event is `DETERMINISTIC` (browser-observed fact) or an `ML_PROPOSAL` an admin confirmed (\$7.5). Score from `appliedWeight`, never `weight`:

```
const scored = events.filter(e =>
  e.status === 'ACTIVE' || e.status === 'CONFIRMED'); // PENDING/DISMISSED/EXPIRED contribute 0
```

With **time decay**, so an early stumble doesn't haunt someone for three hours:

```
riskScore = scored.reduce(
  (s, e) => s + e.appliedWeight * Math.exp(-(now - e.serverTs) / (25 * 60_000)), 0);
```

Dashboard bands: 0–25 green · 26–60 amber · 61–100 orange · 100+ red (auto-surfaced to a proctor).

10.4 The evidence bundle — what you hand to a disciplinary committee

When a student is blocked or flagged, a worker builds an immutable case file:

```
{examId}/{rollNumber}/case/{caseId}/
  report.pdf      # timeline, every event, thumbnails, risk graph
  events.jsonl    # raw event stream
  media/          # every referenced asset
  manifest.json   # {asset: sha256}, signed with the server key
```

The signed manifest lets you prove nothing was edited after the fact — which matters the moment a student contests the result.

11. Feature 8 — Anti-Screenshot / Screen Leak Prevention

11.1 Read this first

In a normal web browser, you cannot prevent screenshots. OS-level capture (Win+Shift+S, macOS Cmd+Shift+4), a phone camera pointed at the monitor, a capture card, OBS, a VM — all of it is entirely outside a web page's reach. Any vendor claiming otherwise for a pure web app is overselling. Plan for **deterrence + detection + attribution**, not prevention.

If you need real prevention, the only route is a native client:

- **Electron lockdown app** — `win.setContentProtection(true)` makes the window render **black** in Windows/macOS screen captures (the same mechanism Netflix uses). Add kiosk mode, blocked shortcuts, process enumeration to spot OBS / AnyDesk / TeamViewer, and clipboard lockdown.
- Roughly 1–2 weeks of work, plus code signing and distribution to 450 machines.
- **Recommendation:** ship the web portal for this exam; build the Electron shell if you repeat this at higher stakes.

11.2 What to implement in the web app — layered deterrence

Layer 1 — Invisible per-student watermark (highest value). Any leaked screenshot traces to a roll number. This is the single most effective measure, because it changes incentives.

```
// apps/web/src/components/Watermark.tsx
export function Watermark({ roll, name }: { roll: string; name: string }) {
  const [t, setT] = useState(() => new Date().toLocaleString());
  useEffect(() => {
    const i = setInterval(() => setT(new Date().toLocaleString()), 30000);
    return () => clearInterval(i);
  }, []);

  const text = `${roll} • ${name} • ${t}`;
  const svg = (w: number, h: number, body: string) =>
    `url("data:image/svg+xml;utf8,${encodeURIComponent(
      '<svg xmlns="http://www.w3.org/2000/svg" width="${w}" height="${h}">${body}</svg>'
    )}");`

  return (
    <div id="wm-root" aria-hidden
      className="pointer-events-none fixed inset-0 z-[9999] select-none overflow-hidden">
      /* faint but visible tiled layer */
      <div className="absolute inset-0 opacity-[0.055]" style={{ backgroundImage: svg(420, 220,
        '<text x='0' y='110' transform='rotate(-28 0 110)' font-family='monospace'
          font-size='17' fill='#000'>${text}</text>'
      ) }} />
      /* near-invisible high-density layer - survives JPEG, invisible on screen */
      <div className="absolute inset-0 opacity-[0.012]" style={{ backgroundImage: svg(150, 80,
        '<text x='0' y='40' font-family='monospace' font-size='8' fill='#000'>${roll}</text>'
      ) }} />
    </div>
  );
}
```

Harden it: render the text from a **server-signed token**, and fire a **CRITICAL** violation if the node is removed or dimmed.

```
new MutationObserver(() => {
  const wm = document.getElementById('wm-root');
  if (!wm || getComputedStyle(wm).display === 'none' || +getComputedStyle(wm).opacity === 0)
    agent.report({ type: 'API_TAMPER_SUSPECTED', severity: 'CRITICAL', payload: { what: 'watermark' } });
}).observe(document.body, { childList: true, subtree: true, attributes: true });
```

Layer 2 — Blur on focus loss / capture-key press. Handles the naive Win+Shift+S user, because the Snipping Tool takes focus.

```
const shield = document.getElementById('capture-shield')!;
const panic = (ms = 1500) => {
  shield.style.opacity = '1';
  setTimeout(() => (shield.style.opacity = '0'), ms);
};

window.addEventListener('blur', () => panic(30_000)); // stays blurred until refocus
document.addEventListener('keydown', e => e.key === 'PrintScreen' && panic());
document.addEventListener('keyup', e => e.key === 'PrintScreen' && panic());
document.addEventListener('keydown', e => e.metaKey && e.shiftKey && '345'.includes(e.key) && panic());
```

```
#capture-shield {
  position: fixed; inset: 0; z-index: 99999; pointer-events: none;
  backdrop-filter: blur(26px); background: rgba(255,255,255,.55);
  opacity: 0; transition: opacity 90ms linear;
}
```

Layer 3 — Clear the clipboard on PrintScreen (Chrome; requires the page to be focused):

```
document.addEventListener('keyup', async e => {
  if (e.key !== 'PrintScreen') return;
  try { await navigator.clipboard.writeText(''); } catch {}
  agent.report({ type: 'PRINTSCREEN_KEY', severity: 'HIGH' });
});
```

Layer 4 — Detect a second display (a common exfiltration path — mirror to a phone or a second monitor):

```
const det = await window.getScreenDetails?(); // needs window-management permission
if (det && det.screens.length > 1)
  agent.report({ type: 'SECOND_DISPLAY_DETECTED', severity: 'HIGH',
    payload: { count: det.screens.length } });

// cheaper fallback, no permission needed:
if ((window.screen as any).isExtended)
  agent.report({ type: 'SECOND_DISPLAY_DETECTED', severity: 'HIGH' });
```

Enforce a single display at preflight — refuse to start until extra monitors are disconnected. This is a real, enforceable win.

Layer 5 — Make a leaked screenshot low-value. - One question per screen — there is no scrollable full paper to grab. - Per-student option shuffle → "the answer is C" is useless to a friend. - Question pools: 450 students drawing from a 900-question bank share very few questions. - Short per-question timers on high-value items.

Layer 6 — CSS/DOM speed bumps. `user-select: none`, `-webkit-touch-callout: none`, `draggable=false`, `@media print { body { display: none } }`. Cheap, stops casual copying, stops nobody determined.

11.3 What each layer actually buys you

Layer	Stops casual	Stops determined	Effort
Watermark + tamper detection	✓ (deters)	✓ attribution	Low
Blur on blur / PrintScreen	✓	✗	Low
Clipboard clear	✓	✗	Trivial
Single-display enforcement	✓	⚠	Low
Question pooling + shuffle	✓	✓ (devalues the leak)	Medium
Electron <code>setContentProtection</code>	✓	✓ (except a phone camera)	High
Anything against a phone camera	✗	✗	—

12. Feature 9 — Issue Flagging & Live Student↔Admin Chat

12.1 Behaviour spec

- A persistent "Report an Issue" button in the exam header — never hidden by the fullscreen overlay.
- The student picks a category (Technical · Question Error · Camera/Mic · Personal Emergency · Other), writes a description, and optionally attaches an auto-captured screenshot.

- Submitting creates or reopens that student's `ChatThread`, keyed to their **roll number**, and pushes it into the admin queue.
- Any of the 5 admin stations can **claim** a thread — atomically, so no two admins reply to the same student.
- Realtime 1:1 chat until resolved. The timer keeps running unless an admin explicitly grants a pause or extra time.

12.2 Socket rooms model

```
student:{rollNumber}  - exactly one student socket
admins                - all 5 stations (queue broadcasts)
thread:{threadId}     - the student + the claiming admin
admin:{adminId}       - direct push to one station
```

12.3 Gateway implementation

```
// apps/api/src/chat/chat.gateway.ts
@WebSocketGateway({ namespace: '/rt', cors: false })
export class ChatGateway {
  @WebSocketServer() server: Server;

  @SubscribeMessage('issue:flag')
  async flag(@ConnectedSocket() sock, @MessageBody() dto: FlagDto) {
    const student = sock.data.student;

    const thread = await this.prisma.chatThread.upsert({
      where: { studentId: student.id },
      create: { studentId: student.id, status: 'OPEN', category: dto.category,
        priority: PRIORITY[dto.category] },
      update: { status: 'OPEN', category: dto.category, priority: PRIORITY[dto.category] },
    });

    await this.prisma.chatMessage.create({
      data: { threadId: thread.id, senderType: 'STUDENT',
        senderId: student.id, body: dto.description },
    });
    if (dto.screenshotKey)
      await this.prisma.chatMessage.create({
        data: { threadId: thread.id, senderType: 'SYSTEM', senderId: 'system',
          body: `[screenshot] ${dto.screenshotKey}` },
      });

    await this.events.log(student.id, 'ISSUE_FLAGGED', 'INFO', { category: dto.category });
    sock.join(`thread:${thread.id}`);

    this.server.to('admins').emit('queue:new', {
      threadId: thread.id, rollNumber: student.rollNumber, name: student.name,
      category: dto.category, priority: thread.priority,
      excerpt: dto.description.slice(0, 120),
      riskScore: await this.risk.get(student.id),
      waitingSince: Date.now(),
    });
    return { ok: true, threadId: thread.id };
  }

  @SubscribeMessage('thread:claim')
  @UseGuards(AdminGuard)
  async claim(@ConnectedSocket() sock, @MessageBody() { threadId }) {
    const adminId = sock.data.admin.id;
```

```

// Redis SET NX = atomic claim across all 5 stations
const won = await this.redis.set(`claim:${threadId}`, adminId, 'EX', 1800, 'NX');
if (!won)
  return { ok: false, reason: 'ALREADY_CLAIMED', by: await this.redis.get(`claim:${threadId}`) };

await this.prisma.chatThread.update({
  where: { id: threadId }, data: { status: 'CLAIMED', claimedBy: adminId } });
sock.join(`thread:${threadId}`);
this.server.to('admins').emit('queue:claimed', { threadId, adminId });

const history = await this.prisma.chatMessage.findMany({
  where: { threadId }, orderBy: { createdAt: 'asc' } });
return { ok: true, history };
}

@SubscribeMessage('chat:send')
async send(@ConnectedSocket() sock, @MessageBody() { threadId, body }) {
  await this.assertParticipant(sock, threadId);
  const isAdmin = !!sock.data.admin;
  const msg = await this.prisma.chatMessage.create({
    data: {
      threadId,
      senderType: isAdmin ? 'ADMIN' : 'STUDENT',
      senderId: isAdmin ? sock.data.admin.id : sock.data.student.id,
      body: body.slice(0, 2000),
    },
  });
  this.server.to(`thread:${threadId}`).emit('chat:message', msg);
  await this.prisma.chatThread.update({ where: { id: threadId }, data: { updatedAt: new Date() } });
}

@SubscribeMessage('thread:resolve')
@UseGuards(AdminGuard)
async resolve(@MessageBody() { threadId, note }) {
  await this.prisma.chatThread.update({ where: { id: threadId }, data: { status: 'RESOLVED' } });
  await this.redis.del(`claim:${threadId}`);
  this.server.to(`thread:${threadId}`).emit('thread:resolved', { note });
  this.server.to('admins').emit('queue:resolved', { threadId });
}
}

```

12.4 Admin superpowers on the same channel

```
@SubscribeMessage('admin:action')
@UseGuards(AdminGuard)
async action(@MessageBody() dto: AdminActionDto) {
  const { rollNumber, action, value } = dto;
  const attempt = await this.svc.byRoll(rollNumber);

  switch (action) {
    case 'GRANT_TIME': // value = minutes
      await this.prisma.attempt.update({ where: { id: attempt.id },
        data: { deadlineAt: addMinutes(attempt.deadlineAt!, value) } });
      break;
    case 'CLEAR_WARNINGS':
      await this.prisma.attempt.update({ where: { id: attempt.id }, data: { warningCount: 0 } });
      break;
    case 'UNBLOCK':
      await this.prisma.attempt.update({ where: { id: attempt.id },
        data: { status: 'IN_PROGRESS', warningCount: 0, blockedReason: null } });
      break;
    case 'BLOCK':
      await this.prisma.attempt.update({ where: { id: attempt.id },
        data: { status: 'BLOCKED', blockedReason: value } });
      break;
    case 'FORCE_SUBMIT': await this.exam.submit(attempt.id, 'ADMIN'); break;
    case 'RESET_SESSION': await this.redis.del(`session:${attempt.studentId}`); break;
    case 'CAPTURE_NOW':
      this.server.to(`student:${rollNumber}`).emit('cmd:capture', { reason: 'ADMIN_REQUEST' }); break;
    case 'START_RECORDING':
      this.server.to(`student:${rollNumber}`).emit('cmd:record', { on: true }); break;
  }

  await this.audit.log('ADMIN', dto.adminId, action, attempt.id, { value });
  this.server.to(`student:${rollNumber}`).emit('exam:state-changed');
}
```

Every one of these writes to `AuditLog`. When a student appeals, you need to show who granted whom extra time.

12.5 Queue UX for 5 stations and 450 students

- Sort by `priority DESC`, `waitingSince ASC`. `PERSONAL_EMERGENCY` and `TECHNICAL` outrank `OTHER`.
- Wait-time badges that turn red past 3 minutes.
- Round-robin auto-assign toggle so the 5 stations split load instead of racing for the same thread.
- Canned replies (`/restart` , `/camera` , `/time`) — with 450 students you will type the same six answers all day.
- A broadcast box: one message to all 450 ("Q17 has a typo; it will be ignored in grading").

13. Feature 10 — Admin Proctor Dashboard

13.1 Screens

Route	Purpose
<code>/admin/live</code>	Grid of 450 tiles: webcam thumb, roll number, risk colour, warnings, connection dot
<code>/admin/review-queue</code>	The primary working screen. Pending ML proposals awaiting a human decision — webcam + screen snapshot side by side, Approve / Record-only / Dismiss (\$7.5.4)
<code>/admin/alerts</code>	Realtime feed of confirmed + deterministic events (already-applied violations)
<code>/admin/queue</code>	Issue/chat queue for the 5 stations
<code>/admin/student/{roll}</code>	Full timeline: events, media gallery, answer progress, chat history, action bar
<code>/admin/exam/{id}</code>	Aggregate: submitted count, average progress, top-risk table, system health
<code>/admin/review</code>	Post-exam adjudication — every flagged event with CHEATING / FALSE_POSITIVE / INCONCLUSIVE
<code>/admin/results</code>	Grading, answer key, export

13.2 Rendering 450 live tiles without melting the browser

```
// Virtualize + lazy thumbnails. Never 450 live video streams.
<FixedSizeGrid columnCount={10} rowCount={45} height={900} width={1600}
  columnWidth={155} rowHeight={125}>
  {({ columnIndex, rowIndex, style }) => {
    const s = students[rowIndex * 10 + columnIndex];
    if (!s) return null;
    return (
      <div style={style} className={cn('rounded border-2 p-1', riskBorder(s.riskScore))}>
        <img src={s.latestThumbUrl} loading="lazy" className="h-16 w-full object-cover" />
        <div className="font-mono text-[10px]">{s.rollNumber}</div>
        <div className="flex justify-between text-[9px]">
          <span>{s.answered}/{s.total}</span>
          <span className={s.warnings ? 'text-red-600' : ''}>⚠️{s.warnings}</span>
          <span className={s.online ? 'text-green-500' : 'text-gray-400'}>●</span>
        </div>
      </div>
    );
  }}
</FixedSizeGrid>
```

- Push **one aggregated snapshot every 5 s** to the `admins` room — not 450 individual events: `ts setInterval(async () => { this.server.to('admins').emit('live:snapshot', await this.svc.dashboardSnapshot(examId)); }, 5000);`
- Thumbnails: the worker writes a 96×72 WebP alongside each stored webcam frame. Serve those, never full frames.
- Fetch a live stream only when an admin **clicks a tile** (opens a WebRTC view — see §17.3 #22).

13.3 Sorting that matters

Default the grid to **risk score descending**. With 450 students and 5 proctors, nobody is visually scanning a grid — they are working a prioritized list.

13.4 Manual unblock by roll number — the release valve

§6.5 terminates students automatically. That is only acceptable because a human can undo it in seconds. This screen is the counterweight to the hard-stop rule, and it should be the fastest page in the admin portal.

`/admin/unblock` — a single search box that takes a **roll number**.

13.4.1 What the admin sees before deciding

Searching a roll number returns everything needed to judge, on one screen:

22BCS1047 · Priya Sharma	●	BLOCKED	14:32:07
Reason: Auto-submitted – 3 screen violations			
Progress: 48/60 answered · risk 34 · 71 min elapsed			
FLAG 1	14:03:11	Tab hidden	[webcam] [screen] 4s
FLAG 2	14:19:48	Fullscreen exit	[webcam] [screen] 2s
FLAG 3	14:32:07	Fullscreen exit	[webcam] [screen] 41s
Connection: 3 heartbeat gaps · longest 38s			
△ 22 other students dropped in the same 60s window at 14:19			
Reason (required) _____			
Restore warnings to: () 0 (•) 2 () keep 3			
Grant extra time: [15] minutes ☒ auto-credit 12 min			
[Unblock & restore] [Keep blocked] [Open chat]			

Two rows on that card do the real work. The **evidence thumbnails per flag** let an admin see in two seconds that flag 3 was a Windows update popup, not a second monitor. And the “22 other students dropped in the same window” line is the tell that this was *your* outage, not the student's cheating — the correlation query in §14.5 computes it, and it turns a judgement call into a fact.

13.4.2 The unblock transaction

```
// apps/api/src/admin/unblock.controller.ts
@Post('unblock/:rollNumber')
@Roles('PROCTOR', 'SUPER_ADMIN')
async unblock(@Param('rollNumber') roll: string, @Body() dto: UnblockDto, @Req() req) {
  if (!dto.reason?.trim())
    throw new BadRequestException('A reason is required – it goes on the permanent record.');
```



```
  return this.db.transaction(async tx => {
    const attempt = await tx.attempt.findFirstOrThrow({
      where: { student: { rollNumber: roll.toUpperCase() }, examId: dto.examId },
      include: { student: true, exam: true },
    });
    if (attempt.status !== 'BLOCKED')
      throw new ConflictException(`${roll} is ${attempt.status}, not blocked.`);

    // 1. how much time did they lose while locked out?
    const lostMs = Date.now() - attempt.blockedAt!.getTime();
    const creditMin = dto.autoCredit ? Math.ceil(lostMs / 60000) : 0;
    const grantMin = (dto.grantMinutes ?? 0) + creditMin;

    // 2. reopen the attempt
```

```

await tx.attempt.update({
  where: { id: attempt.id },
  data: {
    status: 'IN_PROGRESS',
    submittedAt: null, // un-submit
    blockedReason: null, blockedAt: null,
    warningCount: dto.restoreWarningsTo ?? 0, // usually 2 - one strike left
    deadlineAt: addMinutes(attempt.deadlineAt!, grantMin),
    pausedMs: attempt.pausedMs + (dto.autoCredit ? lostMs : 0),
    unblockCount: { increment: 1 },
  },
});

// 3. mark the flags as overturned so they stop counting toward risk
if (dto.voidFlags)
  await tx.proctorEvent.updateMany({
    where: { attemptId: attempt.id, id: { in: dto.voidFlags } },
    data: { status: 'DISMISSED', verdict: 'FALSE_POSITIVE',
      appliedWeight: 0, reviewerId: req.user.id, reviewedAt: new Date(),
      reviewNote: dto.reason },
  });

// 4. let them log back in - clear the denylist and session block
await this.sessions.clearRevocation(attempt.studentId);

// 5. permanent record. Non-negotiable.
await this.audit.log('ADMIN', req.user.id, 'UNBLOCK', attempt.id, {
  rollNumber: roll, reason: dto.reason, grantMin, creditMin,
  restoredWarnings: dto.restoreWarningsTo ?? 0, voidedFlags: dto.voidFlags,
});

this.push(roll, 'exam:restored', {
  message: `A proctor has restored your test. You have ${grantMin} extra minutes. `
    + `Please sign in again with your roll number.`,
  warningsRemaining: attempt.exam.maxWarnings - (dto.restoreWarningsTo ?? 0),
});

return { ok: true, rollNumber: roll, grantMin, creditMin };
}

```

13.4.3 Details that matter in practice

- **Reason is mandatory.** Free text, stored in `AuditLog` forever. It is what you show if someone asks why 30 students were reinstated.
- **Default warnings to 2, not 0.** A genuinely reinstated student keeps one strike in hand; a student gaming the unblock desk doesn't get a clean slate every time. Make it a deliberate radio choice, not a hidden default.
- **Auto-credit the lockout time.** A student blocked at 14:32 and unblocked at 14:44 lost 12 minutes through no fault of their own. Credit it automatically and show the number — this is exactly the "no unfair disadvantage" case.
- **They must sign in again.** The old token was denied and is unrecoverable. Unblocking clears the denial so a fresh login works; it doesn't resurrect the dead session.
- **unblockCount is visible.** Three unblocks on one roll number is a pattern, and the fourth request should get more scrutiny, not less.
- **Bulk unblock exists for outages.** When your ML box or a campus switch dies and 40 students trip flags simultaneously, one admin pastes a list of roll numbers and reinstates them in a single audited action. Build it — the alternative is 40 individual clicks during the worst ten minutes of your day.

```
@Post('unblock/bulk')
@Roles('SUPER_ADMIN') // deliberately narrower than single unblock
async bulkUnblock(@Body() dto: { rollNumbers: string[]; reason: string; grantMinutes: number }) {
  // one AuditLog row per student, one incident id linking them
}
```

- Restrict who can do it. Single unblock: any proctor. Bulk unblock: `SUPER_ADMIN` only. Pair it with the two-person rule (§17.3 #24) if you want blocks *and* unblocks both witnessed.

14. Infrastructure, Deployment & Capacity Planning

14.1 Sizing for 450 concurrent

Component	Spec	Notes
App server	8 vCPU / 16 GB	Nginx + 4 clustered Node instances + Redis
Database	4 vCPU / 8 GB / 100 GB SSD	Postgres 16, <code>max_connections=200</code> , PgBouncer in front
ML server	8 vCPU / 16 GB / 1× T4 or RTX 3060	FastAPI + 2 Uvicorn workers
Storage	500 GB (screenshots) / 2 TB (with video recording)	MinIO on SSD
Bandwidth	~25 Mbps sustained ingest for frames; ~200 Mbps with full recording	Measure before exam day

Socket math: 450 WebSockets is nothing for Node. The load is *media ingest*, not connection count.

14.2 Nginx essentials

```
upstream api {
  least_conn;
  server 127.0.0.1:3001; server 127.0.0.1:3002;
  server 127.0.0.1:3003; server 127.0.0.1:3004;
}

server {
  listen 443 ssl http2;
  server_name quiz.example.edu;
  client_max_body_size 10M; # evidence uploads

  location /socket.io/ {
    proxy_pass http://api;
    proxy_http_version 1.1;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection "upgrade";
    proxy_read_timeout 3600s; # long exams
  }

  location /api/ { proxy_pass http://api; proxy_set_header X-Real-IP $remote_addr; }
  location / { proxy_pass http://127.0.0.1:3000; } # Next.js
}
```

HTTPS is mandatory — `getUserMedia` and `getDisplayMedia` do not work over plain HTTP (except on `localhost`).

14.3 Required security headers

```
// apps/web/next.config.js -> headers()
{
  'Content-Security-Policy':
    "default-src 'self'; img-src 'self' blob: data:; media-src 'self' blob:; " +
    "connect-src 'self' wss://quiz.example.edu; script-src 'self' 'wasm-unsafe-eval'; " +
    "frame-ancestors 'none'; object-src 'none'",
  'X-Frame-Options': 'DENY',
  'Permissions-Policy': 'camera=(self), microphone=(self), display-capture=(self), window-management=(self)',
  'Strict-Transport-Security': 'max-age=63072000; includeSubDomains; preload',
  'Referrer-Policy': 'no-referrer',
}
```

14.4 Reliability requirements — the ones that will save you

1. **Autosave every answer immediately** (400 ms debounce) plus a full state sync every 15 s. A crash costs a student ≤ 15 s.
2. **Resume on reconnect** — `GET /attempt/{id}/state` returns answers, current index, remaining time and warning count; the client rehydrates.
3. **The timer is server-owned.** The client displays `deadlineAt - serverNow`, refreshed on every heartbeat, reconciling clock skew. Never trust client `Date.now()`.
4. **Offline queue** — an IndexedDB buffer for violations and evidence, flushed on reconnect, so a 30-second Wi-Fi drop doesn't lose the audit trail.
5. **Auto-submit at `deadlineAt`** enforced by a server cron sweeping expired attempts — not by the client's timer.
6. **Graceful degradation** — if the ML service dies, the exam continues; frames queue in BullMQ and get analysed late. Proctoring must never be able to take the exam down.

14.5 Progress checkpointing — a system failure must never cost a student marks or minutes

Your infrastructure *will* hiccup during a 90-minute exam with 450 people on it. The design goal is that a student who loses connection, whose laptop dies, or who is caught in a server restart resumes exactly where they were, **with the lost minutes given back**. Two separate problems: don't lose their *answers*, and don't lose their *time*.

14.5.1 Answers — durable on every interaction

Every answer write is an immediate, idempotent upsert. Not batched, not on "next", not on submit.

```
// client - fires on every option click, 400ms debounce for typing
async function saveAnswer(questionId: string, value: AnswerValue) {
  const rev = ++localRevision[questionId];
  ui.setSaveState('saving');
  try {
    await api.put(`/attempt/${attemptId}/answer`, { questionId, value, revision: rev, clientTs: Date.now() });
    ui.setSaveState('saved');
  } catch {
    await idb.put('pending-answers', { questionId, value, revision: rev }); // survives a crash
    ui.setSaveState('offline'); // visible amber pill, never a silent failure
  }
}
```

```

-- server: last-write-wins by revision, so a retry or a reorder can never regress an answer
insert into answers (attempt_id, question_id, value, revision, answered_at)
values ($1, $2, $3, $4, now())
on conflict (attempt_id, question_id) do update
    set value = excluded.value, revision = excluded.revision, answered_at = now()
    where answers.revision < excluded.revision; -- <-- the guard that makes retries safe

```

Three properties make this survive a crash: - **IndexedDB queue**. An answer saved while offline is written to disk in the browser and flushed on reconnect. Closing the laptop lid does not lose it. - **Revision guard**. A delayed retry from 30 seconds ago cannot overwrite a newer answer. Without this, offline replay actively corrupts data. - **Visible save state**. A green "saved 2s ago" pill (\$17.2 #14). Students who can see their work is safe stop refreshing the page, which is itself a major source of incidents.

14.5.2 Time — credit back what the failure cost

The timer is `deadlineAt = startedAt + durationMin + pausedMs`. The `pausedMs` column is what protects the student.

```

model Attempt {
  // ...
  pausedMs      Int      @default(0)      // total credited-back milliseconds
  lastSeenAt    DateTime? // updated by every heartbeat
  unblockCount  Int      @default(0)
  interruptions  AttemptInterruption[]
}

model AttemptInterruption {
  id            String    @id @default(cuid())
  attemptId     String
  startedAt     DateTime
  endedAt       DateTime?
  durationMs    Int?
  cause         String    // CLIENT_NETWORK | SERVER_OUTAGE | BLOCKED | ADMIN_PAUSE | UNKNOWN
  credited      Boolean   @default(false)
  @@index([attemptId, startedAt])
}

```

A `pg_cron` job every 30 seconds opens an interruption for any attempt whose heartbeat has gone quiet, and closes it when they come back:

```

-- open interruptions for the silent
insert into attempt_interruptions (attempt_id, started_at, cause)
select a.id, a.last_seen_at, 'UNKNOWN'
from attempts a
where a.status = 'IN_PROGRESS'
    and a.last_seen_at < now() - interval '30 seconds'
    and not exists (select 1 from attempt_interruptions i
                    where i.attempt_id = a.id and i.ended_at is null);

```

14.5.3 Whose fault was it? — the rule that makes crediting fair

This is the important part, and it's cheap to compute. If many students drop simultaneously, it was your infrastructure. If one student drops, it was their Wi-Fi.

```

-- correlated-outage detector: run in the same 30s cron
with per_minute as (
  select date_trunc('minute', started_at) as minute,
         count(*) as dropped,
         (select count(*) from attempts where status='IN_PROGRESS' and exam_id=$1) as active
  from attempt_interruptions
  where cause = 'UNKNOWN' and started_at > now() - interval '5 minutes'
  group by 1
)
update attempt_interruptions i
  set cause = 'SERVER_OUTAGE'
  from per_minute p
 where date_trunc('minute', i.started_at) = p.minute
       and p.dropped >= greatest(10, p.active * 0.15); -- 15% of the cohort at once = us, not them

```

Then credit automatically, and only for outages you caused:

```

// on reconnect
const gap = Date.now() + interruption.startedAt.getTime();
if (interruption.cause === 'SERVER_OUTAGE') {
  await creditTime(attemptId, gap); // automatic, no one has to ask
  notify(roll, `We had a problem on our side. ${Math.ceil(gap/60000)} minutes have been added.`);
} else {
  await flagForAdmin(attemptId, gap); // one student's own network - a human decides
}

```

Students never have to notice, argue, or file a request to get back time an outage took from them. A single student's home Wi-Fi drop goes to a proctor who can grant time from \$12.4 — because that call genuinely needs judgement, and because auto-crediting individual drops is trivially farmable.

14.5.4 Resume must be a single atomic read

One endpoint returns everything needed to rebuild the exam screen. Anything spread across several calls will eventually rehydrate half-broken.

```

// GET /attempt/:id/state
{
  status: 'IN_PROGRESS',
  serverNow: '2026-09-10T14:44:02.113Z', // reconcile clock skew - never trust Date.now()
  deadlineAt: '2026-09-10T15:19:02.000Z', // already includes pausedMs
  remainingMs: 2100000,
  questionOrder: ['q_8f...', 'q_2a...', ''], // the student's own shuffle, stable
  currentIndex: 47,
  answers: { 'q_8f...': { optionIds: ['o_11...'], revision: 3 }, 'q_2a...' },
  marked: ['q_12...', 'q_31...'],
  warningCount: 2,
  maxWarnings: 3,
  creditedMinutes: 12, // shown to the student explicitly
  proctoringRequired: ['camera', 'screen', 'mic'] // re-request permissions on resume
}

```

Re-granting camera and screen-share **needs a user gesture**, so the resume screen must be a "Resume test" button, not an automatic redirect. Budget for it in the UI or every reconnect becomes a support ticket.

14.5.5 The failure drills to actually run

Write these into the mock exam (§14.6), because untested recovery is not recovery:

Drill	Expected outcome
Kill Wi-Fi for 45 s mid-question	Amber pill, answers queue, flush on return, no data loss
Hard-kill the browser tab, reopen	Resumes at the same question, all answers present
Restart the API mid-exam	Students reconnect; correlated-outage rule credits everyone automatically
Laptop sleeps for 5 minutes	Interruption opened + closed, time credited or flagged per cause
Database failover	Kill switch freezes timers; no attempt auto-submits during the gap
Terminate a student, then unblock	Fresh login works, time credited, warnings restored to 2

14.6 Load test before exam day — non-negotiable

```
// k6 - 450 VUs, ramping
import ws from 'k6/ws';
import http from 'k6/http';

export const options = { stages: [
  { duration: '2m', target: 450 },
  { duration: '20m', target: 450 },
  { duration: '2m', target: 0 },
]};

export default function () {
  const { token } = http.post(`${BASE}/auth/login`,
    { rollNumber: `TEST${__VU}`, password: 'x' }).json();

  ws.connect(`${WSS}/rt?token=${token}`, {}, socket => {
    socket.setInterval(() => socket.send(JSON.stringify({ event: 'hb', data: { t: Date.now() } })), 5000);
    socket.setInterval(() => http.post(`${BASE}/proctor/webcam-frame`, fakeJpeg()), 3000);
    socket.setTimeout(() => socket.close(), 1_200_000);
  });
}
```

Watch: p95 API latency, socket disconnect rate, BullMQ queue depth, Postgres connection count, MinIO write latency.

Then run a full 50-student mock exam with real humans, a week before, running every drill in §14.5.5. Every problem you will have on exam day shows up there first.

15. Build Order / Milestones

#	Milestone	Deliverable	Est.
0	Scaffold	Monorepo, Docker Compose, Prisma migrate, CI	2 d
1	Auth	Roster import, login, session lock, JWT + refresh, admin auth	3 d
2	Exam core	Question delivery (no keys), checkpointing + resume + time credit (\$14.5), server timer, grading	7 d
3	Focus proctoring	Fullscreen lock, visibility/blur, shortcut blocking, 3-flag hard-stop + logout (\$6.5)	5 d
4	Media capture	Webcam + screen share, periodic frames, evidence bundle, MinIO	4 d
5	ML service	FastAPI + YOLO11n + InsightFace, worker, temporal smoothing, risk scoring	5 d
5b	Review queue	Proposal pipeline, admin review cards, claim lock, admission control, precision metrics (\$7.5)	4 d
6	Audio	AudioWorklet RMS, ring buffer, VAD endpoint	2 d
7	Chat & flags	Threads, claim lock, realtime chat, admin actions, audit log	4 d
8	Admin dashboard	Live grid, alert feed, student detail, queue, review, unblock console (\$13.4)	7 d
9	Anti-leak	Watermark + tamper detect, blur shield, display enforcement, question pooling	3 d
10	Hardening	Load test, chaos test, mock exam, runbook, backups	5 d
Total			≈52 dev-days

Minimum viable for exam day if time is short: milestones 0–4 plus 7, plus 5b — the review queue is what makes milestone 5 safe to enable at all. Never launch a brand-new auto-blocking detector on a real exam.

16. Legal, Consent & Privacy

Non-optional if you are recording 450 people's faces, screens and rooms.

1. **Explicit consent screen** before preflight — a checkbox, timestamped, stored on the [Attempt](#) . List exactly what is captured (webcam frames, screen frames, audio on trigger), the retention period, who can view it, and how to contest a flag.
2. **Data minimisation** — store frames, not continuous video, unless risk crosses a threshold.
3. **Retention policy** — non-flagged media auto-deleted at 30 days by a lifecycle job. Flagged evidence: 1 year, or until appeals close.
4. **Access control + audit** — every evidence view logged with admin id, student roll and timestamp. Presigned URLs expire in 5 minutes.
5. **Encryption** — TLS in transit; SSE-S3/SSE-KMS at rest; encrypted DB volumes.
6. **No automated decision from ML — enforced architecturally, not by policy.** Per §7.5, a model detection can only create a zero-weight proposal; a named admin must approve it before it affects a student. Every sanction therefore has a human decision-maker on record. State this in the consent form: it is fairer, it is what makes an appeal answerable, and under DPDP/GDPR-style regimes it keeps you out of "solely automated decision-making" territory.

7. **Accessibility & accommodation** — a documented alternate path (offline/supervised) for students without a camera, on slow links, or with a disability. The admin `GRANT_TIME` action exists for exactly this.
 8. **Bias caveat** — face matching and gaze detection have known demographic error disparities. Use them to *rank for review*, never to decide.
 9. **India-specific (DPDP Act 2023)** — biometric data requires clear notice and purpose limitation; name a grievance officer; honour erasure requests after the retention window.
-

17. Proposed Additional Features

17.1 Stronger anti-cheating

1. **Question pooling + randomised variants** — a 900-question bank, each student draws 60. Two students sitting side by side share ~7% of questions. This beats every technical measure combined.
2. **Numeric parameterisation** — the same question with different numbers per student (`{a}=17` vs `{a}=23`). Colluding students arrive at different answers.
3. **Answer-pattern collusion detection (post-exam)** — cluster students by identical *wrong* answers and submission timing. Classic psychometrics; catches groups no camera ever will. `sql -- pairs sharing more than 8 identical WRONG answers SELECT a.roll, b.roll, COUNT(*) AS shared_wrong FROM graded a JOIN graded b ON a.question_id = b.question_id AND a.chosen = b.chosen AND a.correct = false AND a.roll < b.roll GROUP BY 1, 2 HAVING COUNT(*) > 8 ORDER BY 3 DESC;`
4. **Keystroke & interaction biometrics** — a student typing at 40 WPM for 20 minutes who then pastes a 300-character answer in 0.4 s gets flagged. Cheap to build, hard to fake.
5. **Impossible-timing flags** — repeatedly answering a 3-minute question correctly in 6 seconds.
6. **Second-device side camera** — require the student's phone to run a companion page as a **side-view camera** showing hands and desk. This is the single biggest coverage gain: it kills the "phone in the lap" and "person off-camera" cases a laptop webcam physically cannot see.
7. **Browser extension detection** — probe known ChatGPT/Copilot sidebar extension resources via `chrome-extension://` fetches and flag hits.
8. **VM / remote-desktop detection** — WebGL renderer strings (`llvmpipe` , `VMware SVGA` , `VirtualBox`), suspicious `hardwareConcurrency` , odd screen dimensions.
9. **Canary questions** — 2–3 questions whose text appears nowhere else. If they surface on Telegram mid-exam, you know the paper leaked and roughly when.
10. **Copy-detection honeypots** — invisible text inside question bodies (`position:absolute; left:-9999px`) that rides along with any copied selection. If it shows up in a pasted answer or a leaked screenshot, you have proof of the copy path.
11. **Real gaze estimation** — replace the crude yaw proxy with L2CS-Net or MediaPipe FaceMesh for a genuine "eyes off screen" percentage.
12. **Network fingerprinting** — flag when many students share one public IP (a lab, or one machine proxying for several).

17.2 Better student experience

13. **Mandatory 5-minute mock exam the day before**, on the same stack with the same permissions. Halves exam-day support tickets. Do this one.
14. **Persistent connection-health pill** — green/amber/red with "your last answer saved 2 s ago". Kills the number-one source of anxiety.

15. **Question palette** — answered / unanswered / marked-for-review grid with keyboard navigation.
16. **Auto-reconnect with a visible countdown** instead of a dead page.
17. **Grace-period policy** — the first fullscreen exit in the first 60 seconds is a friendly tutorial, not a warning. Most warning-1 events are honest mistakes.
18. **Low-bandwidth mode** — screen frames to 60 s, webcam to 10 s, recording off. Auto-enable when RTT > 800 ms.
19. **On-screen scratchpad + calculator**, so students don't reach for a phone or another window in the first place.
20. **Accessibility** — font scaling, high contrast, screen-reader-labelled questions, per-student extra time.
21. **Post-exam transparency page** — the student sees their own violation log. Reduces disputes dramatically, because they can see it was recorded fairly.

17.3 Better admin experience

22. **Live WebRTC peek** — click a tile to open a realtime view of that student's camera (SFU via mediasoup, or direct P2P for one-off views). Frames are for the record; live view is for judgement calls.
23. **Auto-triage inbox** — a ranked review queue instead of a wall of alerts. One proctor can meaningfully review ~90 students; 5 proctors on a ranked queue cover 450.
24. **Two-person rule for blocks** — a block above a severity threshold needs a second admin's confirmation. Stops one panicked proctor from ending 30 exams.
25. **Timeline scrubber** on the student detail page — drag through the exam and see webcam frame, screen frame and answer state side by side at that moment. This is what actually resolves an appeal.
26. **Broadcast announcements** to all 450 (typo corrections, time extensions).
27. **Exam-day runbook + kill switch** — one button that pauses all attempts and freezes timers if the network or power fails. Write the runbook before, not during.
28. **Post-exam item analytics** — difficulty, discrimination index, distractor analysis. Tells you which questions were bad, which matters for fairness.
29. **Automated per-student PDF incident report**, ready to hand to a committee.
30. **Shadow mode for new detectors** — run every new detector log-only for one exam, measure its false-positive rate, *then* make it warnable. Never skip this.

17.4 Longer-term platform features

31. **Electron lockdown client** (§11.1) — real screenshot prevention, process scanning, kiosk mode.
32. **Adaptive testing (IRT)** — difficulty adapts per student, so no two students see the same sequence. Anti-cheating and better measurement at once.
33. **Coding / subjective question types** — Monaco editor plus a sandboxed Judge0 runner; rubric-based manual grading UI.
34. **Multi-exam / multi-tenant** — you will have built 80% of a platform; add org accounts and reuse it.
35. **Hash-chained event log** — each `ProctorEvent` stores `hash(prev.hash + payload)`. Cheap, and it makes the audit trail provably unedited.

Appendix A — Reality Check

Things this document deliberately does **not** promise:

- **You cannot stop screenshots in a browser.** §11 is deterrence and attribution. Only a native client gets close, and nothing at all stops a phone camera pointed at the monitor.

- **You cannot stop a second device.** A phone on the desk is invisible to the laptop webcam. Only the side-view camera (§17.1 #6) or physical invigilation addresses it.
- **You cannot prevent inspection of client code.** Everything shipped to the browser is readable. Answer secrecy comes from §5's server-side grading and nothing else.
- **YOLO will produce false positives.** A water bottle reads as a phone; a sibling walking past reads as `MULTIPLE_FACES`. This is *expected*, and §7.5 is built around it: a detection is a proposal with evidence attached, and only an admin's Approve turns it into a violation. Temporal smoothing (§7.4) and the review queue are not optional extras — they are the thing that makes the system defensible.
- **Deterministic signals can still misfire.** An OS popup steals focus; a laptop dies mid-exam. Those *do* auto-warn (§6.3), so set `maxWarnings` generously, make unblocking a one-click admin action, and staff your 5 stations expecting to use it.

The system's real value is not "prevents all cheating" — it is **raising the cost of cheating, recording everything, and giving a human the evidence to make a fair call**. Build for that and you will ship something that works on exam day.